香山昆明湖 V2

自定义向量点积指令

三阶段技术报告合订本

设计、验证与性能评估

三阶段工程技术报告

2026 年 11 月 8 日修订

目录

第一阶段：环境部署与向量执行过程 **1**

1.1 协同验证框架与实验环境 **1**

1.2 部署与构建过程 **2**

1.2.1 资源配置与工具位置 2

1.2.2 固定源码与工具安装 3

1.2.3 集成构建与操作记录 4

1.3 共同启动输入与差分故障定位 **5**

1.4 hello 输出与三类 vadd 结果 **6**

1.4.1 输入配置与结果解释 6

1.4.2 10 月 9 日环境验证 6

1.4.3 10 月 25 日双模型复核 6

1.5 向量执行与波形观察 **7**

1.5.1 10 月 9 日事件与原始截图 7

1.5.2 10 月 25 日含时钟的调试记录 8

1.6 版本与复现索引 **10**

1.6.1 源码与工具版本 10

1.6.2 10 月 25 日复核的产物摘要 11

1.6.3 文件与记录索引 11

1.7 操作日志摘录 **12**

1.7.1 10 月 9 日工作负载构建 12

1.7.2 10 月 9 日正常运行摘要 13

1.7.3 10 月 25 日生产模型输出 13

1.7.4 10 月 25 日 FST 模型输出 14

第二阶段：指令设计与基础测试 **15**

2.1 设计目标与实施范围 **15**

2.2 指令语义与编码 **15**

2.2.1 八项有符号点积与结果布局 15

2.2.2 数值范围与中间位宽 15

2.2.3 编码与合法执行状态 16

2.3 数据通路与弹性两槽控制 17

2.3.1 乘积槽与结果槽 17

2.3.2 时钟、复位与连续流动 17

2.3.3 背压、重定向与事务守恒 18

2.4 RTL 实现与处理器接入 20

2.4.1 关键实现语句 20

2.4.2 译码、配置依赖与写回 22

2.4.3 共享 VFEX4 的派发与门控 23

2.5 基础功能与分层验证 24

2.5.1 正数点积测试 25

2.5.2 负数点积测试 25

2.5.3 边界值点积测试 25

2.5.4 算术数据通路测试汇总 26

2.5.5 响应配对与复查位置 26

2.6 设计取舍与扩展 27

2.6.1 两槽结构与结果保存 27

2.6.2 十六项 INT8 点积的功能进展 28

2.6.3 元素位宽的参数化验证 28

2.7 源码与证据入口 29

第三阶段：协同仿真与性能评估 30

3.1 从功能验证到工程评价 30

3.2 NEMU 参考语义与处理器接入 30

3.3 差分检验与错误检出 31

3.4 全核波形中的点积事务 32

3.4.1 源数据、结果与身份 32

3.4.2 从有效区间到握手边沿 32

3.4.3 输入与接受 33

3.4.4 结果与响应 33

3.4.5 提交与架构目的 34

3.5 等价 RVV 对照与测量方法 **34**
3.5.1 完整窗口与配对统计 35

3.6 硬件微基准结果 **35**

3.7 软件调用与 C8 优化 **36**

3.8 软件接入与复现说明 **37**
3.8.1 固定平台与数据布局 37
3.8.2 编译期选择与调用 37
3.8.3 从函数对象到可执行镜像 39

3.9 完整前向计算的组织与验证 **40**
3.9.1 从点积内核到模型计算链 40
3.9.2 工作量与数据布局 40
3.9.3 完整计算链的验证结果 40

3.10 综合面积、时序与设计成本 **41**

3.11 设计取舍与应用方向 **42**

3.12 补充：三种标准 RVV 实现的完整前向观测 **42**

3.13 评估对象与证据索引 **44**
3.13.1 复现与图像来源 44

第一阶段：环境部署与向量执行过程

本阶段对应赛题的环境部署验证、操作日志和向量加法执行过程，主要结果有三项：一是在 WSL2 中搭建了香山处理器仿真、NEMU 参考模型和测试程序的完整构建与协同仿真环境；二是定位并修复了两端启动代码不一致导致的差分失败；三是三类 vadd 用例共 24 个元素在 DiffTest 下全部通过，并用波形观察了向量加法从执行到提交的过程。

实验分两次进行：10 月 9 日完成环境搭建和首次验证，10 月 25 日复核了英文 hello 输出、两种仿真模型的差分运行和含时钟的波形。第1.6、1.7节集中列出版本信息和日志摘录，供复核使用。自定义 vdot 点积的设计与测试见第二阶段。

1.1 协同验证框架与实验环境

香山 EMU、NEMU 参考模型和裸机程序共同构成验证环境。EMU 模拟硬件执行，NEMU 按指令语义更新参考状态，DiffTest 在指令提交时比较两者的架构状态。裸机程序提供可重复的输入和结果检查；共同启动 flash 负责在复位后跳转到程序入口。hello 检查程序启动与串口输出，vadd 则覆盖向量配置、加载、按元素加法、存储和提交。

硬件侧由 Java 与 Mill 解析 Scala/Chisel 工程，firtool 处理硬件中间表示，Verilator 与宿主 C++ 编译器生成 EMU。目标程序使用 RISC-V GCC 和 binutils 编译、链接并生成 ELF、BIN 及反汇编。NEMU 在宿主侧构建为参考库，由运行参数显式加载。三条构建链的产物在同一次差分运行中汇合，构建记录和运行记录通过产物的 SHA-256 摘要对应起来。表1-1列出各类产物的作用，以及复核时需要查看的主要记录。

表 1-1 构建产物与复核入口

产物	作用	主要记录
香山 EMU	执行 DUT 仿真	硬件源码、DUT 配置、生成命令、编译工具与文件 SHA-256
NEMU 参考库	提供架构状态参考	固定源码、构建配置、实际加载路径与文件 SHA-256
程序 ELF/BIN	执行 hello 和 vadd 用例	ISA/ABI、入口、装载段、反汇编、镜像摘要
共同 flash	建立一致的启动输入	启动汇编、12 字节镜像、反汇编与显式加载参数
运行记录	关联输入、结果及结束状态	原始日志、退出码、波形、事件 CSV 与信号索引

图1-1给出这一协同验证框架的整体结构。上半区为 WSL2 中的三条编译链：香山工程经 JDK、Mill、firtool 和 Verilator 生成 EMU，AM 程序与启动代码经 RISC-V GCC 生成共同的程序 BIN 和 boot-flash，NEMU 源码由宿主编译器构建为参考动态库。下半区为差分运行：EMU 与 NEMU 加载同一镜像，DUT 按架构提交推进参考模型，两端状态送入 DiffTest 比较，串口输出、波形、比较结果和退出状态统一归档。

10 月 9 日实验在 Windows 下的 WSL2 Ubuntu 22.04.5 运行，宿主处理器为 Ryzen 7 7840H，可

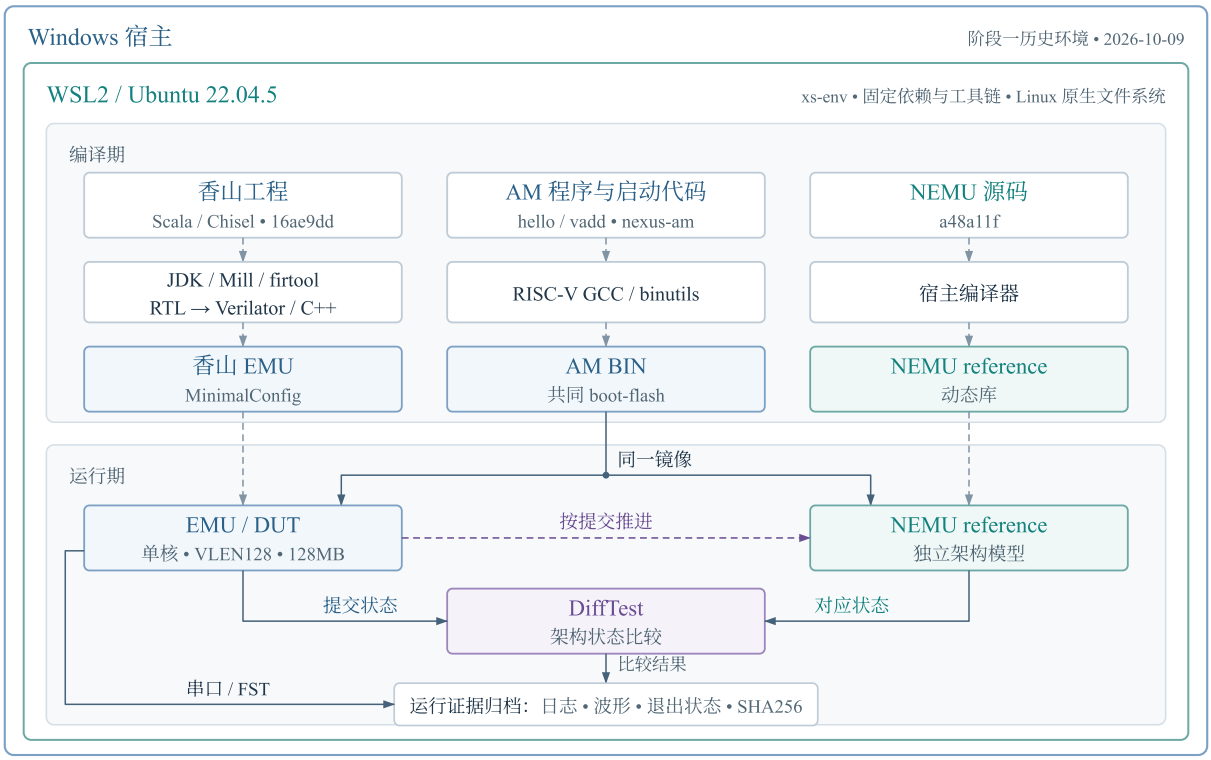


图 1-1 开发环境与软硬件协同验证框架

用物理内存约 27.7GiB。WSL 配置 20GB 内存和 16GB swap，构建与仿真放在 WSL 原生文件系统。DUT 为 MinimalConfig、单核、VLEN128，原波形构建采用 EMU_THREADS=1 和 EMU_TRACE=fst。运行时显式指定 128MB RAM、共同 flash 和固定参考库。

本阶段的源码版本、工具版本和 10 月 25 日复核的产物摘要集中列于本章末尾，两次实验所用的构建分开标注。

1.2 部署与构建过程

环境按本节步骤搭建。10 月 9 日依次完成了 NEMU 参考库、香山 EMU、测试程序和共同启动 flash 的构建，并完成首次差分运行，过程记录见表1-2。

1.2.1 资源配置与工具位置

WSL 资源设置位于 Windows 用户目录的%UserProfile%\.wslconfig：

```
[wsl2]
memory=20GB
swap=16GB
```

保存正在进行的 WSL 工作后，在 PowerShell 执行`wsl --shutdown`，重新进入 Ubuntu 并用`free -h`核对。原机器的交换文件位于 D:\WSL\wsl-swap.vhdx，其他机器按本机磁盘布局配置。内存规划同时覆盖 JVM、硬件生成工具、C++ 编译进程和文件缓存；频繁换页会增加构建时间，因此应根据可用物理内存调整 JVM 堆与编译并发。

Windows 工作树的 Git 检查、提交与推送使用 Windows 原生 Git；Linux/WSL 原生目录使用 Linux Git。Scala/Chisel、RISC-V 程序、EMU/NEMU 和波形处理在 Linux 原生目录执行。源码与

产物跨环境交换时，在接收端核对完整提交号、未提交差异、外部依赖及关键文件 SHA-256，运行输出使用独立目录。

这样安排来自实际遇到的问题：2026 年 10 月 24 日的一次检查中，一次从 WSL 访问 Windows 工作树的检查约 300 秒后超时；核实未提交且没有残留进程或索引锁后，Windows 原生 Git 完成了后续检查与提交。具体过程见协调记录。

1.2.2 固定源码与工具安装

在团队仓库执行`bash team/scripts/prepare-dependencies.sh`，按版本锁初始化 xs-env、NEMU、nexus-am、DRAMsim3 及递归依赖。团队仓库的香山子模块使用`git clone --recurse-submodules` 恢复；NEMU 的 SimPoint 按子模块记录初始化，其余下载依赖按锁定脚本检出。已有目录的版本或已跟踪修改不一致时，先保留现场并核对输入。构建前记录实际仓库根、分支、完整 HEAD、Git common-dir、修改状态与正在使用的输入输出，避免与其他任务共用可写 Git 索引或覆盖冻结产物。

基础工具的安装入口如下。这里的命令用于恢复固定环境，安装后再核对实际工具版本。

```
source team/scripts/activate.sh
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target base
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target llvm
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target mill
sudo apt-get install -y cmake ninja-build gtkwave
```

该入口来自固定 xs-env 的setup-tools.sh。软件源中的补丁版本会变化，环境检查负责记录安装后实际使用的版本。Mill 读取仓库的.mill-version 选择项目版本，Scala、Chisel 和 firtool 按项目解析结果核对。

原 JDK 安装入口使用 Oracle 21/latest 地址。团队的固定安装脚本改用 21.0.12 Linux x64 归档及官方 SHA-256，安装到用户目录：

```
export JAVA_HOME="$HOME/.local/share/graalvm-jdk-21.0.12"
bash team/scripts/install-graalvm.sh
source team/scripts/activate.sh
```

新终端沿用相同 JAVA_HOME。原/opt/graalvm-jdk-21 安装可在版本核对后继续使用；脚本保留版本不同的已有安装。参考资料为Oracle 安装说明、固定归档和官方校验值。

Verilator 从固定提交构建，显式选择已安装的 Clang，并按资源情况控制编译并发：

```
sudo apt-get install -y help2man perl autoconf flex bison
libgoogle-perftools-dev \
  libjemalloc-dev numactl perl-doc zlib1g-dev liblz4-dev libfl2 libfl-dev
mkdir -p "$HOME/riscv-tools"
git clone https://github.com/verilator/verilator.git
"$HOME/riscv-tools/verilator"
cd "$HOME/riscv-tools/verilator"
git checkout ea338be98e1e838d3518809ce8899f85a009963c
unset VERILATOR_ROOT
autoconf
./configure CC=/usr/local/llvm/bin/clang CXX=/usr/local/llvm/bin/clang++
LINK=/usr/local/llvm/bin/clang++
make -j2
sudo make install
```

安装后回到团队仓库根目录，运行 activate.sh 和 check-environment.sh。原环境目录为/home/riscv/riscv-lab/xs-env/XiangShan；activate.sh 将实际团队仓库设为 NOOP_HOME，依据

TEAM_XS_ENV_ROOT 选择 NEMU 和 AM，并设置 LINUX_GNU_TOOLCHAIN=1。向量工作负载构建显式使用`MARCH=rv64gcv`。

1.2.3 集成构建与操作记录

确认源码、工具和输出目录后，依次构建参考库、EMU、目标程序及共同 flash，再执行差分运行：

```
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
bash team/scripts/stage1.sh nemu
bash team/scripts/stage1.sh emu
TEAM_ID= 正式团队标识 bash team/scripts/stage1.sh workload
bash team/scripts/stage1.sh run
```

图1-2展示上述命令对应的执行流程。环境准备、按序构建和 DiffTest 运行依次进行，验证通过后确认结果并归档证据；检查、构建或运行未通过时，分别定位配置与依赖、构建日志或运行差分的原因，修正后返回对应环节重新执行。每一步无论成功或失败，都保留命令、日志、退出码和 SHA-256。

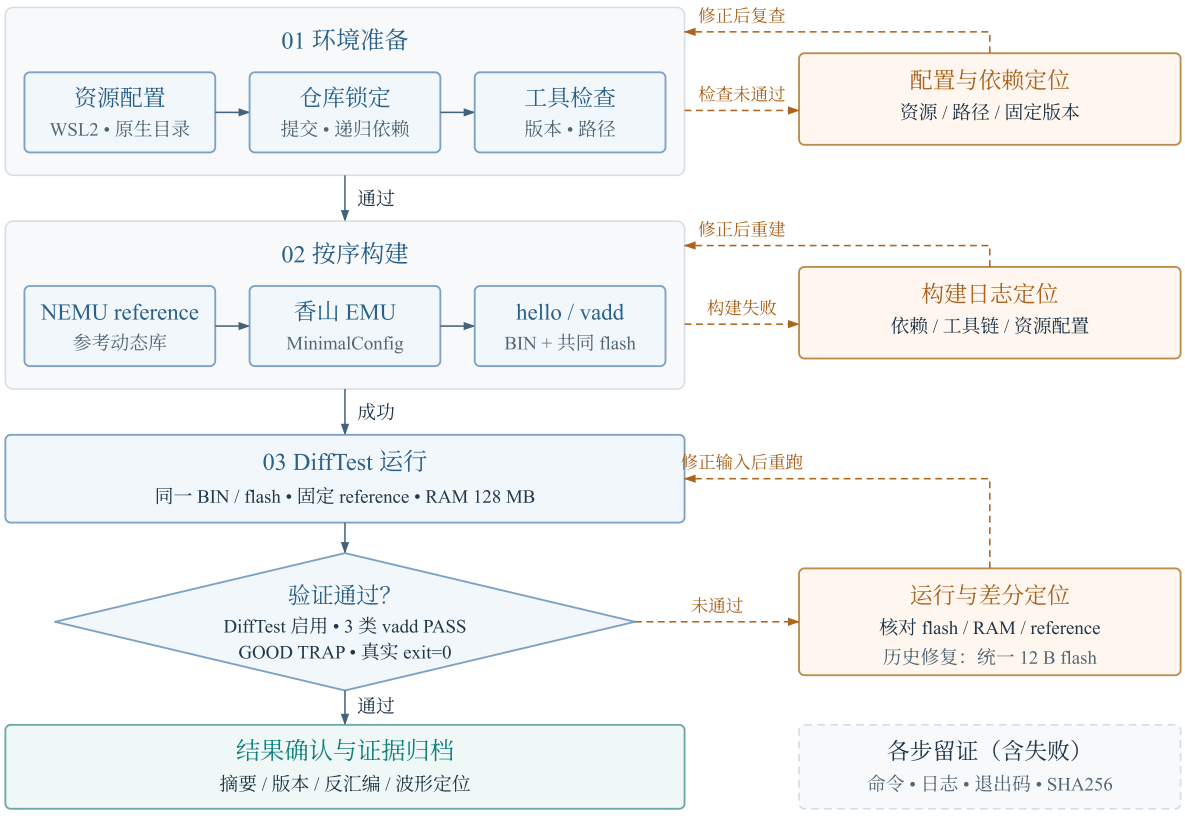


图 1-2 环境部署与验证执行流程

10 月 9 日构建按实际资源调整并行度，过程保留在原始日志中。表1-2摘录当天的主要操作及结果，时间沿用日志的 UTC+8 时区；其中 20:42 的中断与恢复即对应并行度调整。

表 1-2 2026 年 10 月 9 日主要操作记录

时间	操作与结果
20:42:15	中断并恢复 C++ 构建，并行度调整为 4，保留已完成对象。
20:53:32	EMU 构建完成，状态文件记录 exit_code=0。
20:55:26	工作负载与共同 flash 构建完成，记录 WORKLOAD_BUILD_PASS。
21:00:28	正常差分运行结束，退出状态为 0。

编译并发只影响宿主机上的构建速度，与 DUT 核数和 EMU 仿真线程数无关，后两者由模型配置决定。表中 C++ 构建使用的`-j4`与前面 Verilator 安装示例中的`-j2`属于不同步骤。复用对象文件时，源码、工具与生成参数随构建记录一起保存；长作业另记录资源峰值、日志大小、墙钟限额和进程结束状态。

1.3 共同启动输入与差分故障定位

初次运行在 PC0x10000004 出现 t0 寄存器差异，记录的 instrCnt 为 2，尚处于启动代码。比较默认 flash 和反汇编后发现，两端分别使用了 3 条与 6 条启动指令。修复由同一份 boot-flash.S 生成 12 字节 boot-flash.bin，并通过运行参数显式提供给 DUT 与 NEMU。随后两端以共同启动输入进入同一工作负载，在 DiffTest 启用的条件下完成 hello/vadd 检查。

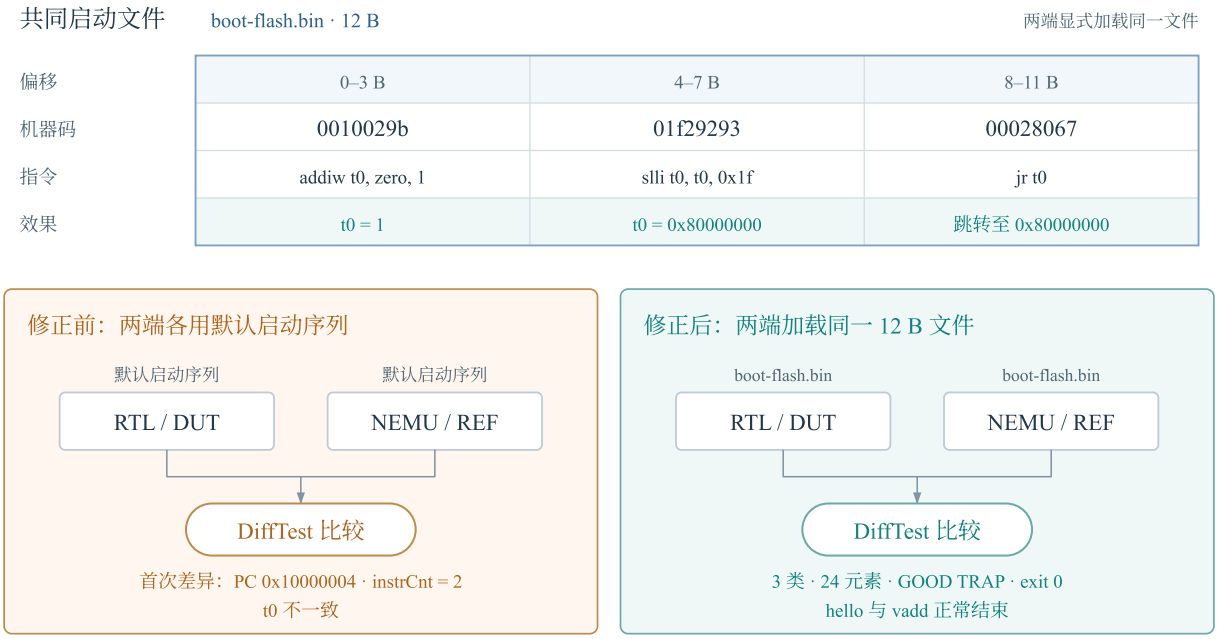


图 1-3 共同启动镜像与差分恢复

ELF 保存符号、装载段和调试资料，BIN 提供程序字节，flash 提供复位后的启动代码。定位波形时，用与被测 BIN 同次构建的反汇编核对 PC 和机器字；链接参数或库布局改变后，应重新建立这组对应关系。flash 中的字节偏移需要按各端的装载地址换算为处理器执行地址。

图1-3对照了修正前后的情况。上部表格逐字节列出共同启动文件：三条指令依次令 t0=1、将其左移为 0x80000000 并跳转至该地址，0–3、4–7 和 8–11 为文件内的字节偏移。下部左侧为修正

前两端各用默认启动序列时的首次差异，右侧为两端显式加载同一文件后的结果：三类共 24 个元素通过比较，程序以 GOOD TRAP 和 exit 0 结束。

1.4 hello 输出与三类 vadd 结果

1.4.1 输入配置与结果解释

vadd 程序设置 SEW8、LMUL1、VL8，执行两次`vle8.v`、一次`vadd.vv`和一次向量存储，再逐字节比较标量参考。三个用例检查基本加法、负数补码和 8 位回绕，合计比较 24 个活动元素。表1-3给出三个用例的输出，分别以日志中的逐元素字节和波形中的低 64 位十六进制两种形式表示。

表 1-3 vadd 输出的两种表示

用例	按元素顺序输出的字节	结果低 64 位
正数相加	9 9 9 9 9 9 9 9	0x0909090909090909
负数补码	254 252 250 248 246 244 242 240	0xf0f2f4f6f8fafcfe
边界回绕	128 127 0 0 0 0 254 254	0xfefe000000007f80

每个元素按模 256 保留低 8 位。例如打印值 254 对应十六进制 FE，按 8 位补码解释为 -2。波形将多个字节合并显示为十六进制，右端为最低字节；日志按元素序号递增输出，因此两种书写顺序应按字节位置对应。VLEN128 给出寄存器宽度，VL8 给出本用例的活动元素数，表中结果覆盖低 64 位的八个元素。

1.4.2 10 月 9 日环境验证

10 月 9 日 hello 输出使用 riscv-vdot-team，身份状态为 placeholder，显示的 IPv4 为 10.198.93.97，对应当时 WSL 宿主。正常记录包含 HELLO_VADD_PASScases=3lanes=24、DiffTest 启用标记、GOOD TRAP 和退出状态 0；整程序统计为 16557 条指令、42962 周期，覆盖启动、UART 和结果校验。图1-4为当次运行结束时的终端输出，可以看到身份行、三类 vadd 的 PASS 结果、DiffTest 启用标记、GOOD TRAP 以及退出状态 0；输出中的身份与地址按原实验保留。

1.4.3 10 月 25 日双模型复核

10 月 25 日（UTC+8）的复核以个人标识 LiZufan001 构建英文 hello 程序，输出明确标注地址来自构建宿主：

```
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
```

同一 BIN 分别在封存生产 EMU 和独立 FST 模型上运行，均加载普通 NEMU 参考库并启用 DiffTest。两次运行都通过三组、24 个元素检查，命中 GOOD TRAP 并正常退出；每次为 16621 条指令、43249 周期。这里的周期同样对应完整 hello/vadd 程序。表1-4列出两次运行的实际起止时间与结果，时间取自各自的运行回执。

2026 年 10 月 9 日原始运行的终端输出

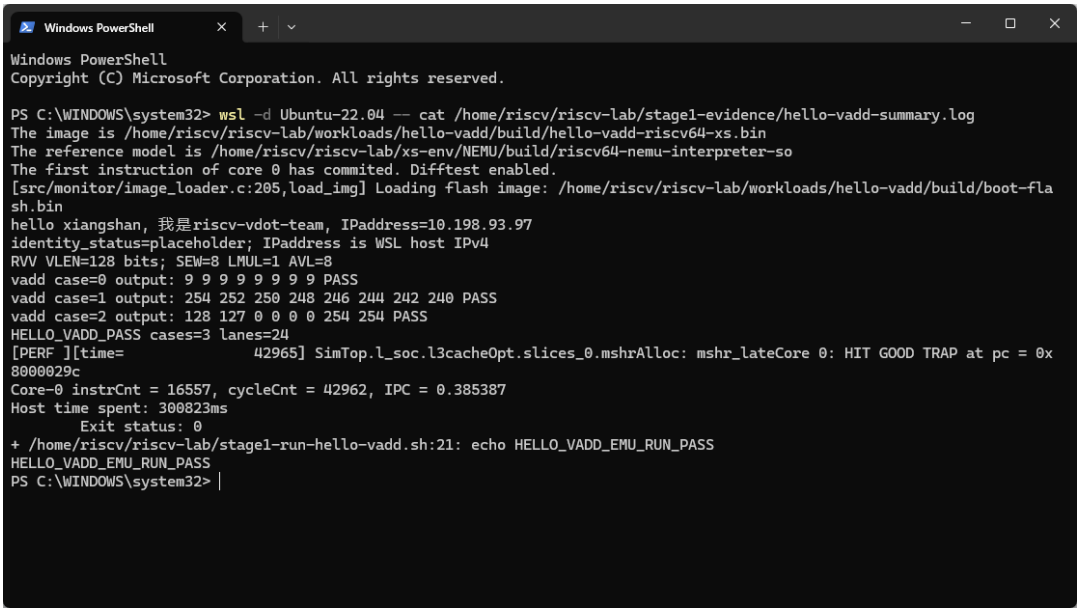


图 1-4 hello 与 vadd 正常结束输出

表 1-4 10 月 25 日双模型运行记录（UTC+8）

模型	实际起止时间	运行结果
生产 EMU	01:09:21.574716–01:10:18.357888	DiffTest 通过，GOOD TRAP，exit 0
独立 FST 模型	01:28:57.974329–01:31:24.009122	DiffTest 通过，GOOD TRAP，exit 0

新 FST 波形模型复用了已有构建的 1807 个输入文件，并逐个核对；运行结束后，原件和副本均未改变。发布源码重新编译得到的 BIN 和 flash 与实际执行输入逐字节相同，原始 ELF 和发布 ELF 分别留存。对应验收记录、生产运行回执和FST 运行回执均随证据保存。

1.5 向量执行与波形观察

向量指令经取指、译码、重命名与调度进入执行单元。两个`vle8.v`提供输入，`vadd.vv`产生结果，随后经过写回与 ROB 提交。本文用提交 PC0x80000138、机器字 0x021101d7 及`vecwen=1`，`wdest=3`关联三类用例，并分别记录结果输出和架构提交两个观察点。

1.5.1 10 月 9 日事件与原始截图

表1-5沿用原 FST 提取 CSV 的事件坐标，列出三类用例在执行输入、结果输出和 ROB 提交三处的位置以及输出低 64 位。局部截图未显示时钟；截图中的 ps 只是波形工具的显示单位，数值为仿真记录的时间坐标，不代表实际时间。本表只用于定位事件，指令时延与性能在第三阶段用专门的测量窗口评估。

表 1-5 10 月 9 日 vadd 事件坐标

用例	输入	输出	提交	输出低 64 位
正数	19625	19626	19631	0x0909090909090909
负数	25460	25461	25466	0xf0f2f4f6f8fafcfe
边界	33009	33010	33015	0xfefe000000007f80

图1-5把表1-5的坐标画成时间线。三类用例都在输入后 1 个周期给出结果，再经 5 个周期完成 ROB 提交；每个用例的三个事件都属于同一条 vadd 指令。

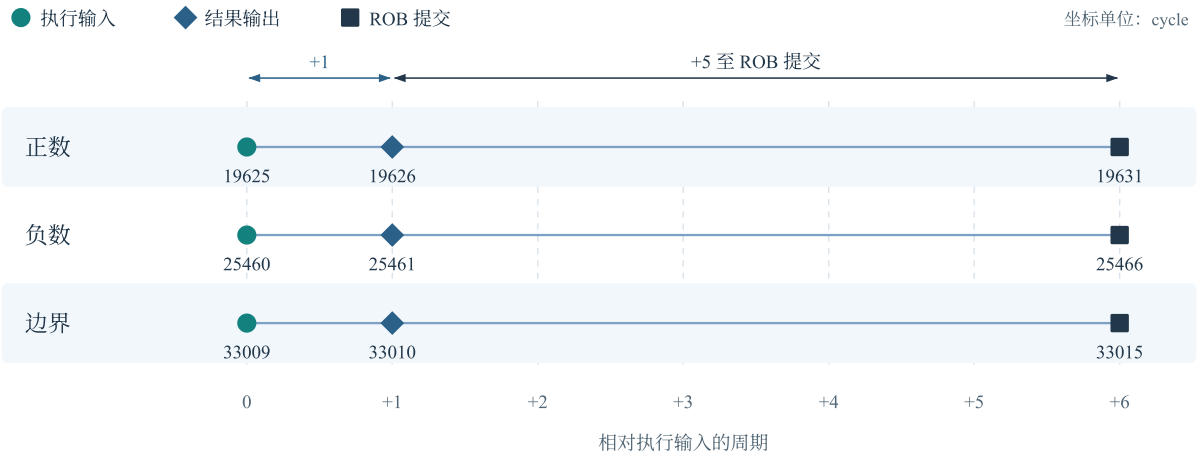


图 1-5 vadd 输入、输出与提交事件时间线

图1-6是正数用例的原始 GTKWave 截图：子图（a）的游标位于 19626 处的输出事件，三个事件的位置同时可见；子图（b）、（c）分别给出 19625 输入游标和 19631 提交游标处的信号值。图1-7为负数与边界用例的输出波形，两者的输出低 64 位分别为 F0F2F4F6F8FAFCFE 和 FEFE00 0000007F80，与表1-3一致。

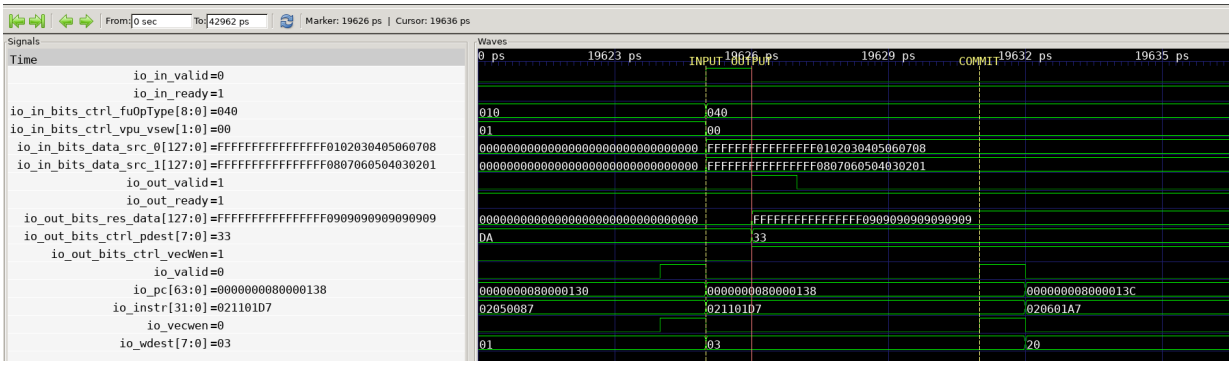
原完整 FST 路径记录为/home/riscv/riscv-lab/stage1-evidence/hello-vadd.fst，大小 220295598 字节。复核可先从提交 PC 定位同次反汇编，再根据 CSV 打开波形窗口，最后联合查看时钟、valid/ready、操作数、结果及事务标识。信号层级与位宽由 evidence/wave-signals.json 提供。

1.5.2 10 月 25 日含时钟的调试记录

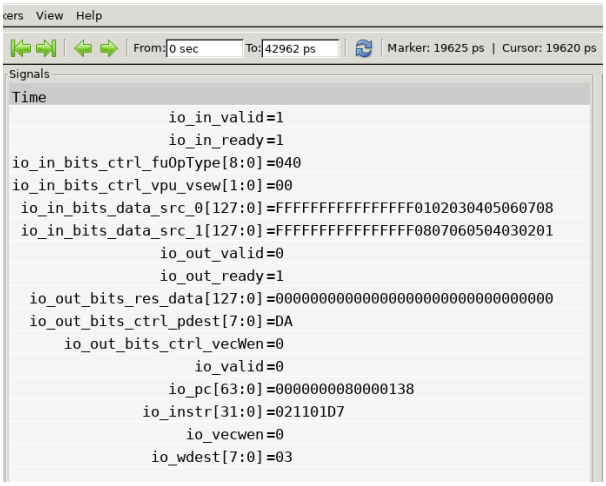
10 月 25 日的 FST 波形模型保留实际时钟信号，最终选择两个 VialuFix、八个提交槽和 127 个信号，检查 33249 个完整上升沿，得到三次 FU 输入、三次 FU 输出和三次 DPI-C 提交观测，共九条事件。三个运算均在 exus_2.VialuFix 发生，同时检查了 exus.VialuFix。

采样取每个上升沿之前、时钟低电平期间的稳定值，周期字段取当时的 DifftestTrapEvent.io _cycleCnt，FST 原始时戳另存。DPI-C 是仿真模型调用 C 函数的接口，提交记录经过 Rob.scala 中`delay=3` 的观察路径，表内 DPI-C 列因此表示提交接口的观测时刻。按上述规则得到的九条事件列于表1-6，表中同时给出每笔运算的 ROB flag、value 和目的物理寄存器。

（a）输出握手：游标 19626 ps，三个事件位置均可见



（b）输入游标 19625 ps



（c）提交游标 19631 ps

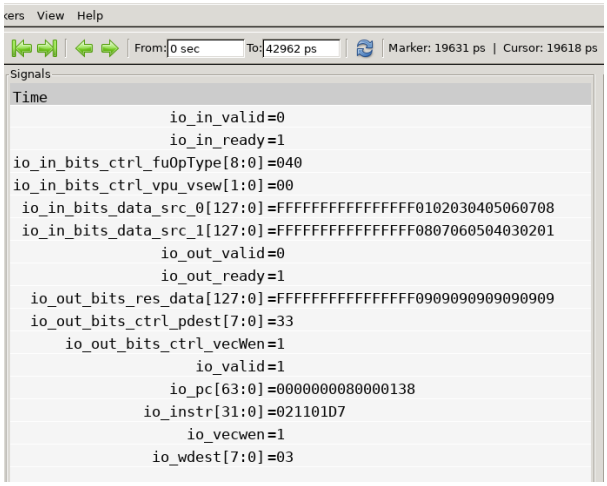
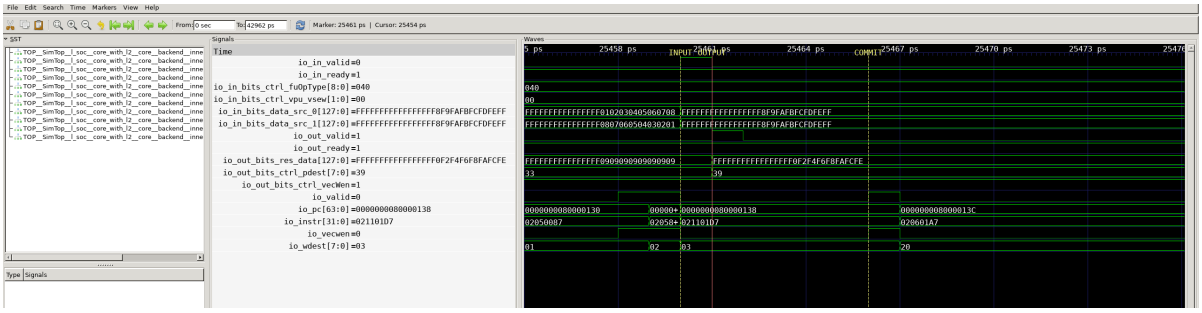


图 1-6　vadd 正数用例的执行与提交波形

（a）负数用例：输出游标 25461 ps



（b）边界用例：输出游标 33010 ps

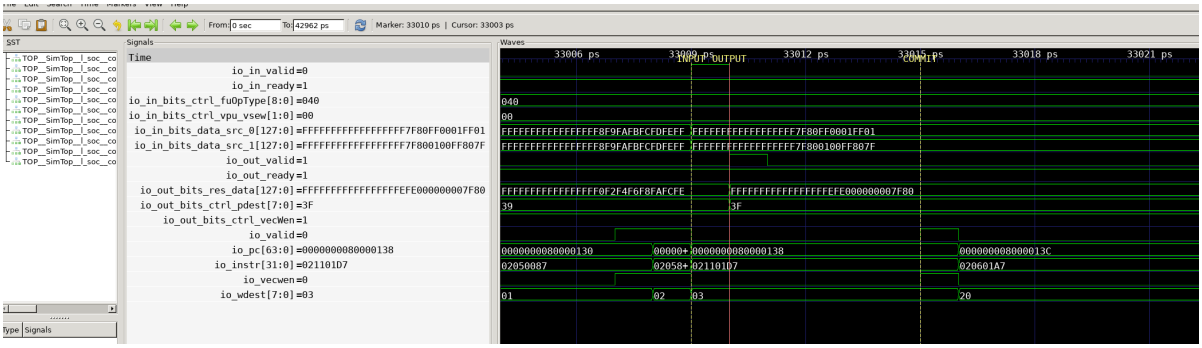


图 1-7　vadd 负数与边界用例结果波形

表 1-6 10 月 25 日按时钟沿采样的事件记录

用例	FU 输入 timer	FU 输出 timer	DPI-C 观测 timer	ROB flag/value/pdest
正数	19525	19526	19531	1 / 13 / 51
负数	25526	25527	25532	0 / 21 / 57
边界	33193	33194	33199	1 / 25 / 63

三类数值结果与表1-3一致。完整采样定义、信号清单、原始时戳及摘要保存在wave.json；只含所选信号的 FST 可直接用 GTKWave 查看。接口与观察路径分别对应固定源码的VIAluFix 包装层和Rob.scala。

工具检查共 24 项，覆盖日志完整性、实际退出状态、输入文件摘要、关键结果，以及缺 FU、缺半周期、未知有效操作数和错误事务标识等反例。首次 FST 构建的工具路径问题和首版提取器遗漏无数字后缀 FU 的记录均保留；修正提取范围后，从同一 FST 完成最终九事件核对。这 24 项检查针对的是记录和提取工具本身，硬件实际运行的仍是上述三组 vadd 用例。

1.6 版本与复现索引

本节列出两次实验所用的源码、工具和产物，供复现和核对使用。

1.6.1 源码与工具版本

表1-7列出 10 月 9 日部署时固定的源码版本，完整递归依赖见 evidence/submodules-after-build.lock.txt。

表 1-7 10 月 9 日源码版本锁

组件	提交号
xs-env	b0b506481e839a5df8a580e76f5aa246dc60fe7c
香山基线	16ae9ddcda54fc9a2fddffad73174cf793ac7814
NEMU	a48a11f208ff81ad765591d83271d1b339fb3861
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
DRAMsim3	1d2a9bf6da8e050d975ef2669c42faa93943f489
DiffTest	62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0
yunsuan	cadd3c2f43096e253d61296b33ac697be8354e29

表 1-8 10 月 9 日工具版本

工具	版本
Oracle GraalVM JDK / javac	21.0.12
LLVM / Clang	19.1.7
Mill	项目版本 0.12.3

表 1-8 10 月 9 日工具版本（续）

工具	版本
Scala / Chisel	2.13.15 / 6.6.0
firtool	1.62.1
Verilator	5.052，提交 ea338be98e1e838d3518809ce8899f85a009963c
RISC-V GCC / GNU binutils	11.4.0（riscv64-linux-gnu-前缀）/ 2.38
宿主 GCC / G++	11.4.0
Make / CMake / Ninja	4.3 / 3.22.1 / 1.10.1
Python / GTKWave	3.10.12 / 3.3.104

表1-8为同一环境实际使用的工具版本，完整版本输出见 evidence/tool-versions.txt；Ubuntu 软件包清单保留原环境的软件来源。新环境按所需工具安装并核对版本；硬件改动后的验证结果与对应的构建产物一起记录。

1.6.2 10 月 25 日复核的产物摘要

10 月 25 日复核所用硬件构建的源码提交为 34bae6d4acceac939f958199dae53c14bc1f6183。表1-9列出这次运行主要产物的 SHA-256，摘要直接对应 evidence/stage1-current-20261025/中的运行和验收记录。

表 1-9 10 月 25 日复核主要产物的 SHA-256

对象	SHA-256
生产 EMU	ff840528eb1009e94906eefb3773ca321050f309553543ab26f0db23c410e30f
FST 模型	1ff3f457ff582d0c3155a15dafa1fa60b8275d9356e29a0b7806c9d3915ce9f7
普通 NEMU 参考库	8af118e5519db7ef6001010a2d547707c5d47630412e80abfc3f343bfef987c9
共同程序 BIN	d81437b44473cef2b0c5777330e13fd076567d4fc1a4296be35d971e51a4dcad
共同 flash	a3360212cf7bb784119c1719c9258362f8567803601b2359aee2c104864fa994
完整 FST（181143183 字节）	cf8abe9104fb955c7dc4bf1efda6588a61060fce33857b9ddc6d5020421be299
选通信号 FST（393288 字节）	4c9f442704871010026654fd9d60854024b270a83be6354be5dee3ff25df24e1

1.6.3 文件与记录索引

运行源码位于 team/workloads/hello-vadd。10 月 9 日证据归档位于 team/evidence/stage1-20261009，10 月 25 日复核的证据位于 team/evidence/stage1-current-20261025；报告源码的 evidence/目录保存用于本文的对应日志与索引。完整 FST 按记录中的路径和摘要定位。报告目录中各附件记录的用途见表1-10。

表 1-10 报告附件索引

报告目录中的记录	用途
evidence/workload-build.log	工作负载、flash 构建命令与成功状态。
evidence/hello-vadd-summary.log	10 月 9 日身份、三组结果、DiffTest 和退出摘要。
evidence/rendered/nemu-build.txt	NEMU 构建日志的可读副本。
evidence/emu-build-20261009-204228.log	EMU 构建与并行恢复过程。
evidence/tool-versions.txt	工具版本原始输出。
evidence/submodules-after-build.lock.txt	递归依赖完整提交锁。
evidence/vadd-commit-events.csv	配置、加载、vadd 及后续指令的提交记录。
evidence/vector-fix-alu-events.csv	三组向量 ALU 输入与输出记录。
evidence/wave-signals.json	10 月 9 日信号层级、位宽与标识。
evidence/stage1-current-20261025/production-run/run.json 与 trace-run/run.json	10 月 25 日两次运行的命令、输入前后摘要、退出状态和日志摘要。
evidence/stage1-current-20261025/wave/wave.json	10 月 25 日的采样定义、127 个信号、九条事件及波形摘要。
evidence/stage1-current-20261025/acceptance.json 与 RESULT.md	10 月 25 日复核的输入等价、24 项工具检查与完整复现索引。

复核时先检查运行回执和通过摘要，再核对程序、参考库与 flash 的 SHA-256，最后用反汇编和事件 CSV 定位波形。详细操作入口见阶段一记录、环境部署方法和10 月 25 日复核的复现说明。

1.7 操作日志摘录

本节保留主要运行摘要与构建完成片段；完整构建日志、版本锁和事件 CSV 随源码包提供，路径见表1-10。

1.7.1 10 月 9 日工作负载构建

```
+ /home/riscv/riscv-lab/stage1-build-workload.sh:20: make ARCH=riscv64-xs
MARCH=rv64gcv -j1
# Building hello-vadd [riscv64-xs] with AM_HOME
{/home/riscv/riscv-lab/xs-env/nexus-am}
+ CC main.c
# Building lib-am [riscv64-xs]
+ AS src/nemu/common/mainargs.S
+ AR -> build/am-riscv64-xs.a
# Building lib-klib [riscv64-xs]
# Creating binary image [riscv64-xs]
+ LD -> build/hello-vadd-riscv64-xs.elf
+ OBJCOPY -> build/hello-vadd-riscv64-xs.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:21: riscv64-linux-gnu-gcc -c
-march=rv64gc -mabi=lp64d boot-flash.S -o build/boot-flash.o
+ /home/riscv/riscv-lab/stage1-build-workload.sh:22: riscv64-linux-gnu-objcopy
-O binary -j .text build/boot-flash.o build/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:23: riscv64-linux-gnu-objdump
-d build/boot-flash.o

+ /home/riscv/riscv-lab/stage1-build-workload.sh:28: echo WORKLOAD_BUILD_PASS
```

```
WORKLOAD_BUILD_PASS
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: rc=0
++ /home/riscv/riscv-lab/stage1-build-workload.sh:1: date -Is
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: printf 'exit_code=%s
end=%s\n' 0 2026-10-09T20:55:26+08:00
```

1.7.2 10 月 9 日正常运行摘要

```
The image is
/home/riscv/riscv-lab/workloads/hello-vadd/build/hello-vadd-riscv64-xs.bin
The reference model is
/home/riscv/riscv-lab/xs-env/NEMU/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image:
/home/riscv/riscv-lab/workloads/hello-vadd/build/boot-flash.bin
hello xiangshan, 我是 riscv-vdot-team, IPaddress=10.198.93.97
identity_status=placeholder; IPaddress is WSL host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
[PERF ][time=               42965] SimTop.l_soc.l3cacheOpt.slices_0.mshrAlloc:
mshr_lateCore 0: HIT GOOD TRAP at pc = 0x8000029c
Core-0 instrCnt = 16557, cycleCnt = 42962, IPC = 0.385387
Host time spent: 300823ms
        Exit status: 0
+ /home/riscv/riscv-lab/stage1-run-hello-vadd.sh:21: echo
HELLO_VADD_EMU_RUN_PASS
HELLO_VADD_EMU_RUN_PASS
```

1.7.3 10 月 25 日生产模型输出

```
emu compiled at Oct 24 2026, 09:22:45
Using simulated 32768B flash
use /home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/b
oot-flash.bin as flash bin
Using simulated 128MB RAM
The image is /home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workloa
d/build/hello-vadd-riscv64-xs.bin
The reference model is /home/lizufan/riscv-lab/t17-clean-20261024-0854/support-
01/nemu/reference/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image: /home/lizufan/ri
scv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin

[src/monitor/image_loader.c:250,load_img] Read 12 bytes from file /home/lizufan
/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin to
0x0xf0000000
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
Core 0: HIT GOOD TRAP at pc = 0x800002a4
Core-0 instrCnt = 16621, cycleCnt = 43249, IPC = 0.384309
Seed=0 Guest cycle spent: 43253 (this will be different from cycleCnt if emu
loads a snapshot)
Host time spent: 56742ms
```

1.7.4 10 月 25 日 FST 模型输出

```
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
Core 0: HIT GOOD TRAP at pc = 0x800002a4
Core-0 instrCnt = 16621, cycleCnt = 43249, IPC = 0.384309
Seed=0 Guest cycle spent: 43253 (this will be different from cycleCnt if emu
loads a snapshot)
Host time spent: 145975ms
```

第二阶段：指令设计与基础测试

2.1 设计目标与实施范围

点积把两组数据逐项相乘，再将乘积求和，是量化矩阵运算的重要计算步骤。本设计在昆明湖 V2 已有向量路径中增加八项 INT8 点积指令，将乘法与归约组合为专用数据通路，同时复用处理器的译码、寄存器重命名、调度、写回和精确提交机制。

通用构建默认采用单请求实现，弹性两槽实现作为可选优化保留。本章重点介绍两槽实现的算术结构、背压和取消处理，并通过正数、负数及边界值测试展示结果。第三阶段的周期和成本数据均来自八项两槽受测版本；十六项扩展的最新功能验证列在本章末尾。本章内容对应赛题 2.1 的设计说明、2.2 的 RTL 实现和 2.3 的三类基础测试。主要结果是：指令按赛题语义实现，正数、负数和边界值三类测试的完整 128 位结果均与期望一致；两槽实现接收请求后最早 2 拍给出结果，每拍可以接收一笔新请求。

2.2 指令语义与编码

2.2.1 八项有符号点积与结果布局

本指令采用 128 位向量寄存器（VLEN=128）、8 位元素（SEW=8）和单寄存器分组（LMUL=1）；有效元素数为 8（VL=8），起始执行索引为 0（vstart=0）。指令读取两个源寄存器的低 8 个字节，将每个字节解释为有符号整数，计算

$$S = \sum_{i=0}^{7} \mathrm{int8}(vs1[8i+7:8i])\,\mathrm{int8}(vs2[8i+7:8i]).$$

目的低 32 位保存 S 的二补码，高 96 位写零。运算从零开始累加，读取指令执行前的源值，因此支持目的与源寄存器重合，也支持使用 v0 作为普通源或目的。每个源以 128 位向量传入数据通路，八项算术使用其中低 64 位的 8 个元素。

图2-1汇总了指令字段、源操作数取位和结果布局。custom-0 编码给出两个源寄存器和一个目的寄存器；两源各取低 64 位的 8 个有符号字节，同编号字节两两相乘后求和；目的低 32 位保存和的补码，高 96 位写零。以 $S = -8$ 为例，完整结果为 0x00000000_00000000_00000000_FFFFFFF8。

2.2.2 数值范围与中间位宽

有符号 8 位数的范围为 -128 至 127，单个乘积范围为 $[-16256, 16384]$，八项和范围为 $[-130048, 131072]$。乘积使用 16 位；按二项、四项、八项归并时，精确位宽依次增加为 17、18、19 位。正端点 131072 比 18 位有符号数的最大值大 1，因此极值用例能直接检验归约过程中是否保留了完整进位。

归约结果先符号扩展为 32 位，再与 96 位零拼接。负数的符号由低 32 位字段表达，例如 -8 的低 32 位为 `0xfffffff8`，完整 128 位结果仍以 96 个零作为高位。也就是说，19 位用于保证求和精确，32 位是结果槽实际保存的宽度，128 位是写回向量寄存器的架构宽度。



图 2-1 八项点积指令字段与完整结果布局

2.2.3 编码与合法执行状态

指令采用 custom-0 编码，`MATCH=0x0200000b`、`MASK=0xfe00707f`，各字段取值见表2-1。汇编记法为 `vdot.vv vd, vs2, vs1`。

表 2-1 vdot 指令的编码字段

字段	位段	含义或取值
funct6	31:26	`000000`
vm	25	`1`
vs2、vs1	24:20、19:15	两个向量源寄存器编号
funct3	14:12	`000`
vd	11:7	向量目的寄存器编号
opcode	6:0	custom-0，`0x0b`

例如 `vdot.vv v10,v8,v9` 对应 `0x0284850b`。程序通过经过校验的编码宏或机器字生成该指令。

向量功能启用、vill=0、vm=1 以及本节的 SEW、LMUL、VL 和 vstart 共同确定合法状态，四种 vta/vma 策略组合均适用。合法执行完成一次完整目的写回，并保持 VL、vtype、vlenb、vxrm、vxsat 和 vcsr。非法请求携带 illegal-instruction 异常完成，同时关闭向量与 v0 目的写使能，由处理器的精确异常机制处理。

2.3 数据通路与弹性两槽控制

2.3.1 乘积槽与结果槽

点积数据通路包含 8 个有符号 8 × 8 乘法器、乘积槽 P、归约网络和结果槽 R。P 保存 8 个 16 位乘积及这笔请求的控制、性能信息；R 保存 32 位结果及同一请求的身份。结果从 R 输出时再补入 96 位零，这样 R 只需保存 32 位，写回接口仍是完整的 128 位。

图2-2给出这条数据通路的结构。8 个乘法器的 16 位乘积存入 P，组合归约网络按 4 个 17 位、2 个 18 位、1 个 19 位逐级相加得到精确和，符号扩展为 32 位后存入 R，输出时再与 96 位零拼接成 128 位结果。图下部为事务身份通路：ROB、目的寄存器、写使能、异常信息和性能字段存入 pCtrl 与 pPerf，P 转入 R 时整包移入 rPacket；若 P 中已锁存非法指令标记，选择器以零代替结果，并关闭目的写使能。

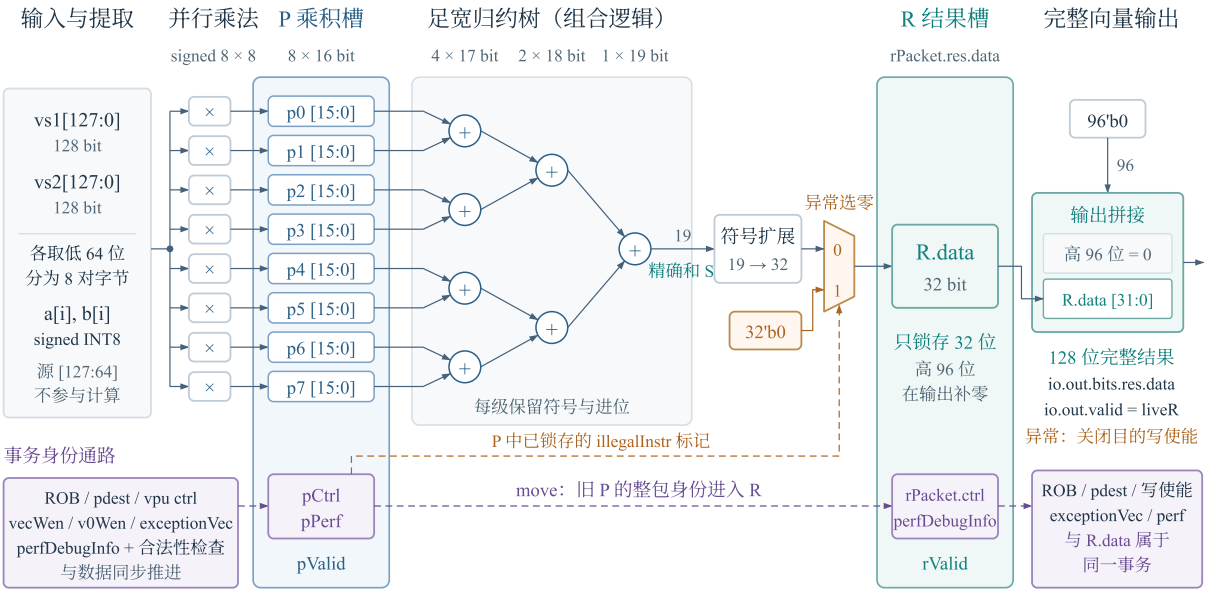


图 2-2 八项点积的乘积槽、归约网络和结果槽

两槽的有效位表示各自是否保存一笔请求。重排序缓冲区（ROB）编号、目的物理寄存器、向量或 v0 写使能、异常信息以及性能字段与数据同时移动，使写回始终能够找到正确的目的。输出停顿时，R 中已有的请求保持不变，不受新输入影响。

2.3.2 时钟、复位与连续流动

VDot 功能单元（FU）接入共享执行单元 VFEX4，后文将 VFEX4 称为父级。实现沿用功能单元所在的单时钟域和父级复位连接。P、R 在时钟上升沿更新；复位清除两槽有效位，并抑制复位期间的输入和输出握手。在父级选择并允许接收 VDot、独立请求连续供给、下游持续就绪且没有复位或重定向时，请求于 E0 进入 P，E1 完成归约并进入 R，E2 是最早的输出握手沿。连续请求的发起间隔为 1 拍。表2-2列出 P、R 与下游状态的典型组合及对应的本拍动作。

图2-3以四笔连续请求为例展示逐拍流动，图中 P、R 两行为时钟沿前的状态。下游持续就绪时，#1 在 cycle 10 被接收，cycle 11 位于 P，cycle 12 位于 R 并完成响应；在 cycle 12 这一拍，接收 #3、#2 由 P 迁入 R、#1 从 R 响应三件事同时完成。由此得到 1 拍的发起间隔和接收后 2 拍的最早响应。

表 2-2 两槽状态与本拍动作

P 状态	R 状态	下游状态	本拍动作
空	空	任意	接受新请求到 P
有效	空	任意	原 P 进入 R，同时可接收下一请求
空	有效	反压	保持 R，P 可接收一笔请求
有效	有效	反压	保持两槽，暂停接收
有效	有效	就绪	原 R 输出、原 P 进入 R、新请求进入 P

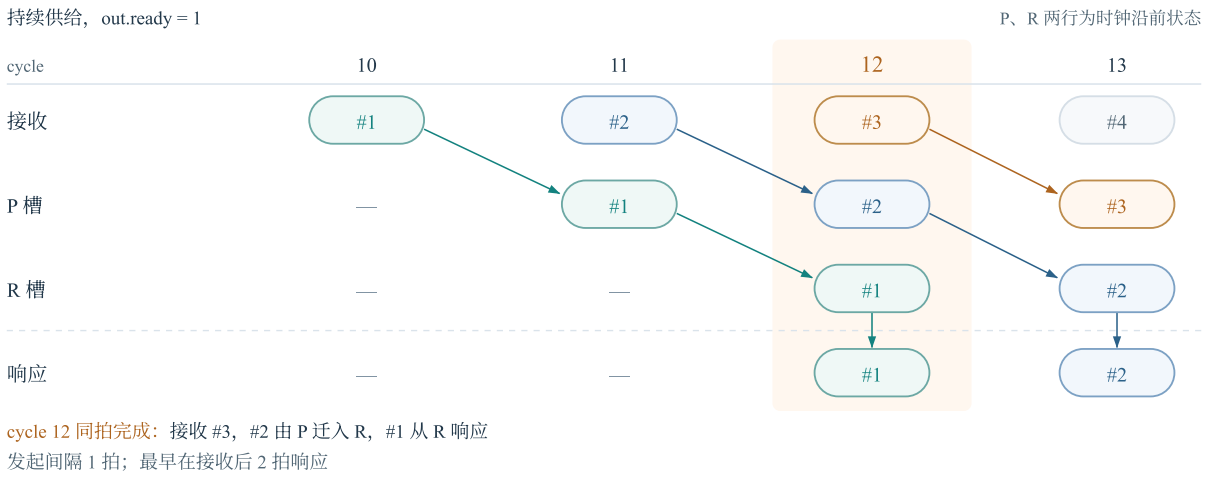


图 2-3 连续请求在两槽中的逐拍流动

全核运行的观察器日志也记录到了这种同拍重叠，如图2-4所示：10986 周期同时记录了 ACCEPT seq2、MOVE seq1 和 RESPONSE seq0，分别对应新请求进入 P、原 P 推进至 R 和原 R 响应。

FU 能否接收取决于自身是否有空槽，但新请求何时派发仍由父级根据当前选择和共享资源占用决定。

时延指一笔请求从被接受到产生响应所用的拍数，发起间隔指相邻两笔请求被接受的间隔。两槽结构支持连续独立点积；程序的最终周期还由数据依赖、调度、访存和写回共同决定。FU 通过 valid/ready 报告完成，下游反压时响应相应顺延。

2.3.3 背压、重定向与事务守恒

R 有效而下游未就绪时，数据、目的和控制包一起保持。P 可利用空槽再接收一笔请求；若两槽均占用且 R 仍受阻，输入 ready 撤销，直到 R 输出或取消腾出空间。

图2-5给出一次实际背压过程，A、B 分别为序号 1321 和 1322 的请求。A 在 +2 拍进入 R，下游从 +2 到 +8 连续 7 拍未就绪，期间 R 保持 A 的结果与目的身份，P 中的 B 也保持不变；+9 拍下游恢复就绪，A 完成一次响应，B 同拍由 P 迁入 R。此时 FU 的 in.ready 为 1，父级的 in.ready 仍为 0。

重定向按两个槽各自的 ROB 年龄独立判断取消，保留较老的幸存请求，并在该拍暂停新请求接收。被取消的 R 停止输出，被取消的 P 释放待归约状态；复位的优先级最高。ROB 环回场景使

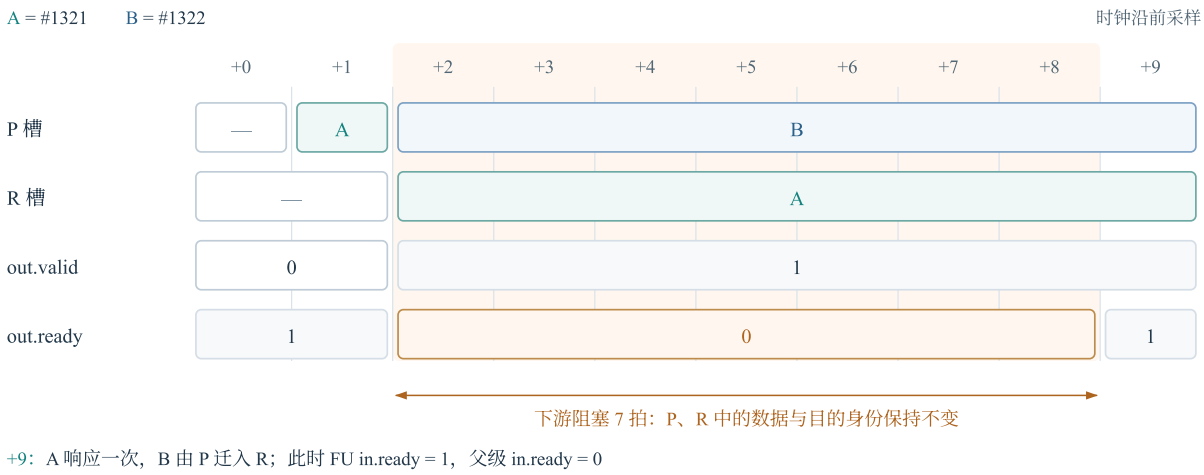
两槽观察器的历史日志摘录（非 GTKWave 波形）

```
S2-05 SUPPORTING EVIDENCE: VDOT TWO-SLOT EVENT LOG EXCERPT
Historical 0f clean-02, dense-batch128, 2026-10-27 (UTC).
Observer event log excerpt; this is NOT a waveform screenshot.
Selected fields copied from the source lines listed below; fields omitted.
Source: /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/dense-batch128/run.log
Source SHA256: 18f472d75feef0378bec71c64793ffd2297bc37c47a5b501597f222da174b98f

line 146 cycle=10984 O2_STATE
  pvalid=0 pseq=0 rvalid=0 rseq=0 occupancy=0 invalid=1 inready=1 outvalid=0 outready=1 move=0
line 148 cycle=10984 O2_ACCEPT
  seq=0 rob=1:31 pdest=110 vl=8 illegal=0 vecwen=1
line 149 cycle=10985 O2_STATE
  pvalid=1 pseq=0 rvalid=0 rseq=0 occupancy=1 invalid=1 inready=1 outvalid=0 outready=1 move=1
line 151 cycle=10985 O2_ACCEPT
  seq=1 rob=1:32 pdest=111 vl=8 illegal=0 vecwen=1
line 152 cycle=10985 O2_MOVE
  seq=0 rob=1:31 pdest=110 illegal=0 vecwen=1
line 153 cycle=10986 O2_STATE
  pvalid=1 pseq=1 rvalid=1 rseq=0 occupancy=2 invalid=1 inready=1 outvalid=1 outready=1 move=1
line 155 cycle=10986 O2_ACCEPT
  seq=2 rob=1:33 pdest=112 vl=8 illegal=0 vecwen=1
line 156 cycle=10986 O2_MOVE
  seq=1 rob=1:32 pdest=111 illegal=0 vecwen=1
line 157 cycle=10986 O2_RESPONSE
  seq=0 rob=1:31 pdest=110 data=0x00000000000000000000000000020000 vecwen=1

At cycle 10986: ACCEPT seq=2, MOVE seq=1, RESPONSE seq=0.
Source result.json reports O2_DENSE_ELASTIC_AND_WINDOWS_PASS and exit_code=0.
Original batch UTC capacity audit FAIL retained; see R5/G5 acceptance RESULT.md.
```

图 2-4　连续事务的同拍重叠记录



+9：A 响应一次，B 由 P 迁入 R；此时 FU in.ready = 1，父级 in.ready = 0

图 2-5　背压期间的数据与目的身份保持

用完整指针年龄关系确定先后。

图2-6给出四种取消与复位场景，均假设下游未就绪（out.ready=0），且重定向当拍不接收新请求。只取消 P 时，R 中的 B 保留，继续等待下游就绪；只取消 R 时，P 中的 A 推进到 R；两槽同时被取消时都变为无效；reset 与 redirect 同拍时以 reset 为准，两槽清空。

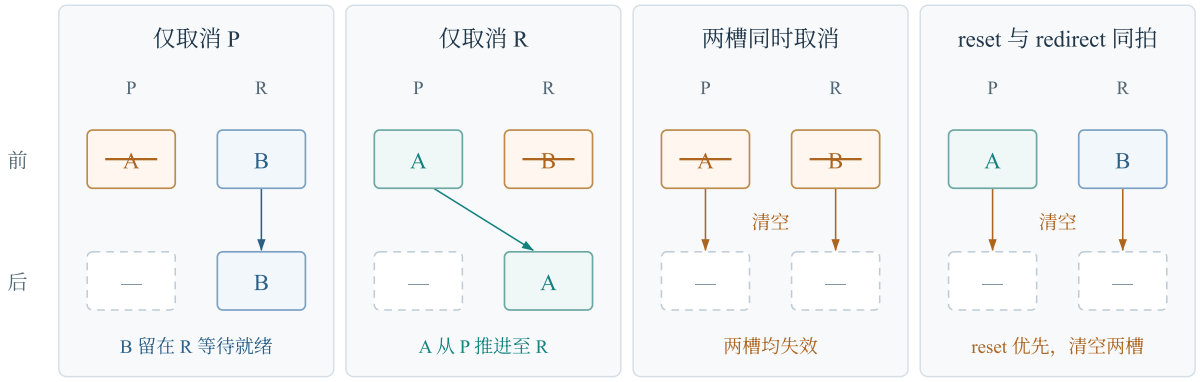


图 2-6 部分取消、全部取消与复位恢复

每次验证在统一时钟坐标下检查

$$N_{\text{accept}} = N_{\text{response}} + N_{\text{cancel}} + N_{\text{reset}} + N_{\text{inflight}}, \qquad 0 \le N_{\text{inflight}} \le 2.$$

该关系将每笔已接受请求归入响应、取消、复位丢弃或当前占用之一，并在运行结束时检查两槽排空。每笔响应还核对完整 128 位结果、ROB 和目的身份。

图2-7综合了正常推进与取消、复位处理。A 部分以 (P.valid, R.valid) 表示状态，标出没有 flush 和 reset 时各状态之间的转移及触发事件；B 部分给出取消与复位的判定顺序：reset 优先清零两槽，未复位时按各自 ROB 匹配取消，幸存请求仍可移动或输出，flush 有效期间禁止新请求进入；C 部分说明接收、移动和输出三件事可以在同一边沿重叠。

2.4 RTL 实现与处理器接入

2.4.1 关键实现语句

算术部分位于 src/main/scala/xiangshan/backend/fu/vector/VDotDatapath.scala，乘积和每级归约按有符号类型计算，并逐级保留进位，对应的源码摘录见图2-8。wrapper/VDotElastic.scala 保存 P、R 的控制和数据，图2-9依次摘录了其中的 32 位结果槽寄存器、握手与取消条件，以及状态更新和原 P 内容转入 R 的逻辑。

下面摘示从 P 转入 R 的关键语句：

```
when(move) {
  rPacket.ctrl := pCtrl
  rPacket.perfDebugInfo := pPerf
  rPacket.res.data :=
  Mux(pCtrl.exceptionVec.get(ExceptionNO.illegalInstr),
    0.U(32.W), reduce.io.result(31, 0))
}
```

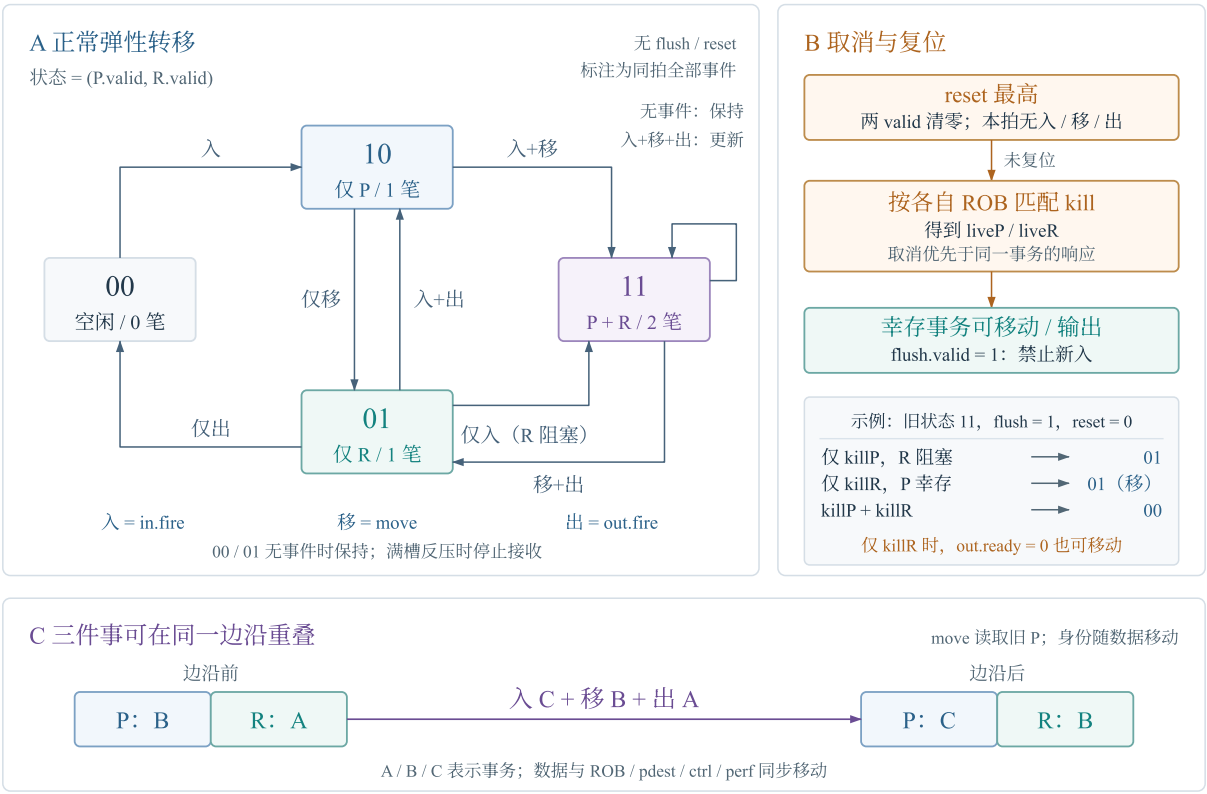


图 2-7　两槽同拍推进与取消、复位处理

（a）受测版本的 VDotProducts 源码

```
class VDotProducts extends RawModule {
  val io = IO(new Bundle {
    val vs1 = Input(UInt(128.W))
    val vs2 = Input(UInt(128.W))
    val products = Output(Vec(8, SInt(16.W)))
  })

  for (lane <- 0 until 8) {
    val a = io.vs1(8 * lane + 7, 8 * lane).asSInt
    val b = io.vs2(8 * lane + 7, 8 * lane).asSInt
    io.products(lane) := a * b
  }
}

/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
```

（b）受测版本的 VDotReduce 源码

```
/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
  val io = IO(new Bundle {
    val products = Input(Vec(8, SInt(16.W)))
    val result = Output(UInt(128.W))
  })

  private val sums17 = Wire(Vec(4, SInt(17.W)))
  private val sums18 = Wire(Vec(2, SInt(18.W)))
  private val sum19 = Wire(SInt(19.W))

  for (pair <- 0 until 4) {
    sums17(pair) := io.products(2 * pair) +& io.products(2 * pair + 1)
  }
  for (pair <- 0 until 2) {
    sums18(pair) := sums17(2 * pair) +& sums17(2 * pair + 1)
  }
  sum19 := sums18(0) +& sums18(1)

  private val result32 = Wire(SInt(32.W))
  result32 := sum19
  io.result := Cat(0.U(96.W), result32.asUInt)
}

/** Combinational arithmetic top for independent verification.
  * The future functional-unit wrapper instantiates the same two submodules
  * separately, with registers between them; this top has no cycle protocol.
  */
```

图 2-8　八项点积算术实现摘录

（a）槽寄存器与 32 位结果状态

```
    require(!cfg.writeFflags && !cfg.writeVxsat && !cfg.flushPipe && !cfg.replayInst)

    val pipelineStatus = IO(Output(new VDotElasticStatus))

>   private val pValid = RegInit(false.B)
    private val rValid = RegInit(false.B)
    private val pProducts = Reg(Vec(8, SInt(16.W)))
    private val pCtrl = Reg(chiselTypeOf(io.out.bits.ctrl))
    private val pPerf = Reg(chiselTypeOf(io.out.bits.perfDebugInfo))
    // H1 stores only the architectural low word; the public result stays 128 bits.
    // Keep each slot's complete control/perf identity on the same transfer edge.
    private val rPacket = Reg(new Bundle {
      val ctrl = chiselTypeOf(io.out.bits.ctrl)
      val res = new Bundle {
        val data = UInt(32.W)
      }
      val perfDebugInfo = chiselTypeOf(io.out.bits.perfDebugInfo)
    })

    pipelineStatus.pValid := pValid
    pipelineStatus.rValid := rValid
    pipelineStatus.occupancy := pValid.asUInt +& rValid.asUInt

    private val illegalInput = vill || vsew =/= VSew.e8 || vlmul =/= VLmul.m1 ||
      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
```

（b）握手与取消条件

```
PS C:\WINDOWS\system32> wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -Pattern 'private val rReady
=' -Context 5,12

      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
    private val killP = pValid && pCtrl.robIdx.needFlush(io.flush)
    private val killR = rValid && rPacket.ctrl.robIdx.needFlush(io.flush)
    private val liveP = pValid && !killP && !reset.asBool
    private val liveR = rValid && !killR && !reset.asBool
>   private val rReady = !liveR || io.out.ready
    private val move = liveP && rReady
    private val pReady = !liveP || rReady

    // Parent admission is enforced by its atomic child dispatch. The FU also
    // suppresses every new accept on redirect/reset, including unrelated flush.
    io.in.ready := pReady && !io.flush.valid && !reset.asBool
    io.out.valid := liveR
    io.out.bits.ctrl := rPacket.ctrl
    io.out.bits.perfDebugInfo := rPacket.perfDebugInfo
    io.out.bits.res := 0.U.asTypeOf(io.out.bits.res)
    io.out.bits.res.data := Cat(0.U(96.W), rPacket.res.data)
```

（c）状态更新与原 P 内容转入 R

```
PS C:\WINDOWS\system32>
                        > wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -SimpleMatch 'pValid := (live
P && !move) || io.in.fire' -Context 2,16

    acceptedCtrl.exceptionVec.get(ExceptionNO.illegalInstr) := illegalInput

>   pValid := (liveP && !move) || io.in.fire
    rValid := (liveR && !io.out.fire) || move

    when(io.in.fire) {
      pProducts := products.io.products
      pCtrl := acceptedCtrl
      pPerf := io.in.bits.perfDebugInfo
    }
    when(move) {
      // Reads pre-edge P even when a new request replaces it at this same edge.
      rPacket.ctrl := pCtrl
      rPacket.perfDebugInfo := pPerf
      rPacket.res.data := Mux(pCtrl.exceptionVec.get(ExceptionNO.illegalInstr),
        0.U(32.W), reduce.io.result(31, 0))
    }

    // Read-only handles for the separate G1/G2 observation harness. These carry
```

图 2-9　两槽状态与控制实现摘录

这些赋值读取本拍边沿前的 P 内容，因此同拍接收的新请求会进入 P，原请求的结果和身份则进入 R。公开输出使用：

```
io.out.bits.res.data := Cat(0.U(96.W), rPacket.res.data)
```

结果寄存器的保存位宽为 32 位，高 96 位由输出组合逻辑补零。

2.4.2　译码、配置依赖与写回

精确编码匹配后，译码器将 vdot 归入向量运算，生成单个微操作并建立两个向量源依赖。实际 VL 通过重命名后的依赖传递，SEW、LMUL 和其他向量控制随指令保存。这样，多条向量配置与运算交错时，每条点积仍使用与自己对应的配置。

旧目的值不参与累加，译码去除相应的旧目的依赖；向量和 v0 的目的分配、异常信息及最终写回沿用现有处理器机制。选择两槽构建时显式启用 `VDOT_O2_ELASTIC=1` 和 `VDOT_T14_FUSED_INPUT=1`，并采用对应配置清单。工程同时保留通用单请求路径，便于按构建版本比较和回退。

2.4.3 共享 VFEX4 的派发与门控

VDot 与整数 DIV、浮点 DIV/SQRT 共用 VFEX4 父级。父级根据选择、实际占用和被选功能单元的 ready 进行原子派发，使父级和子单元接受的是同一笔请求。VDot 两槽排空后可以切换到 DIV；DIV 在途期间保存自己的 ROB 身份和占用状态。输出按唯一有效完成来源选择共享写回。

图2-10是这一共享结构的简化示意。派发与互斥逻辑按选中的 FU 接收请求，三个功能单元各有独立的时钟门控，完成结果经共享写回从唯一有效来源输出；占用与控制逻辑汇总各单元的状态，约束派发，并产生取消和时钟使能。图中省略了部分逐信号连线。

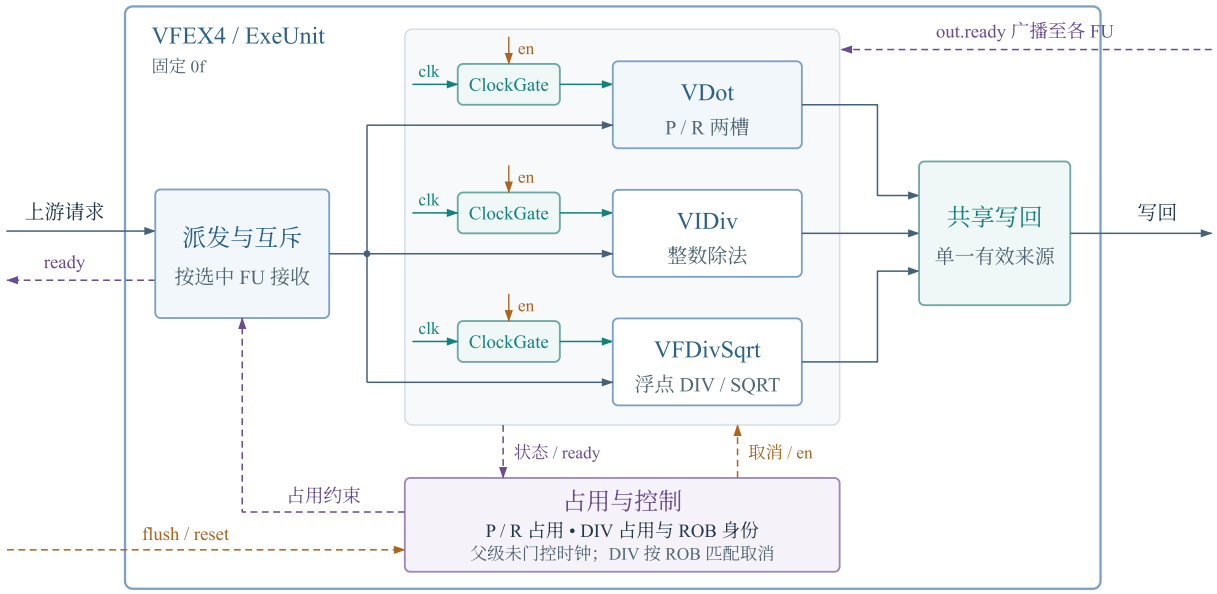


图 2-10 VFEX4 中的派发、资源互斥与共享写回

派发许可同时检查资源占用、请求选择以及 flush 和 reset。上游 ready 由许可与被选 FU 的 ready 共同决定，输入 valid 也仅送往被选单元，因此父级接受与该 FU 接受必须发生在同一边沿。ready 的返回路径按选择信号取值，不能将各 FU 的 ready 直接作或运算。

VDot 占用由 `pValid` 或 `rValid` 表示；DIV 侧还保留在途 ROB 和占用状态，接受后持续保持，直到输出握手或匹配取消。图2-10中的输出 ready 向各 FU 广播，但输出 valid 最多一路有效，共享写回按该唯一有效来源同时选择结果与身份。

时钟使能覆盖存活请求、当前输入接受和必要的取消、复位清理。对应验证包含满槽背压、部分取消、ROB 环回、关钟后的复位恢复，以及 VDot 和两类 DIV 之间的切换。受测的 DIV 取消修复在匹配请求撤销后释放时钟保持，使父级能够进入空闲并接受后续工作。

图2-11给出两段实测过程。上半部分是 VDot 排空后切换到 DIV：旧 R 输出、VDot 排空和 DIV 接收依次相隔 1 拍，整数 DIV 与浮点 DIV 从接收到响应分别用 10 拍和 8 拍。下半部分是取消清理后的时钟变化：DIV 在 +0 拍接收、+1 拍完成取消清理，+2 至 +4 拍父级全部关钟，+5 拍重新接收 VDot，+7 拍给出响应。也就是说，取消清理完成后 DIV 不再保持时钟，父级可以完全关钟，之后再正常切换回 VDot。

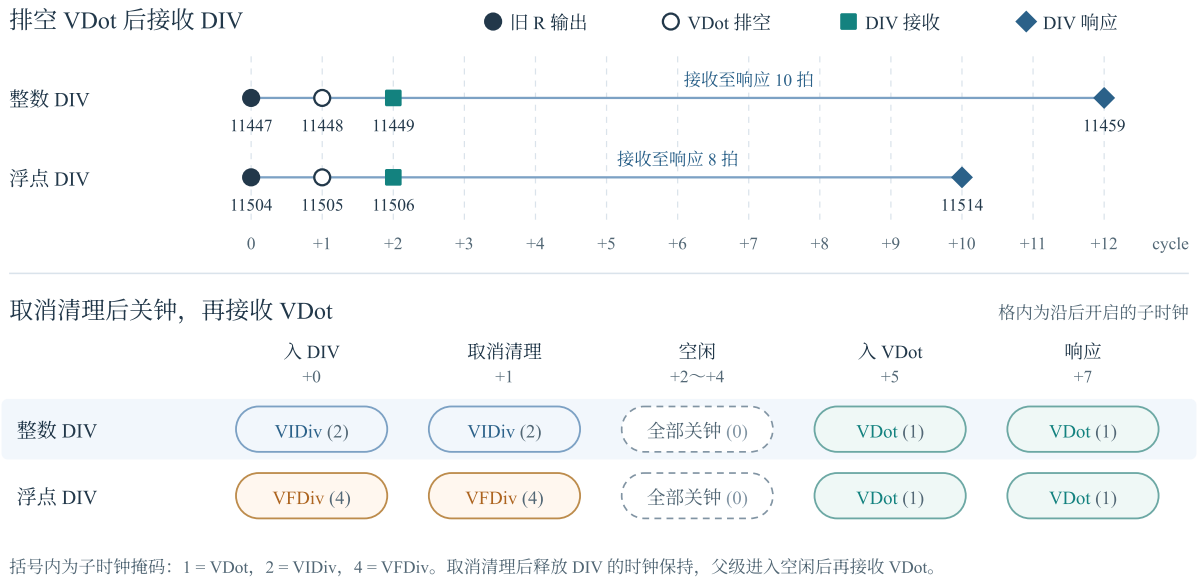


图 2-11　共享资源切换、取消与时钟保持

2.5　基础功能与分层验证

验证首先用独立数学计算确定完整 128 位结果，再依次检查算术 RTL、带握手的功能单元、真实父级和全核执行。算术测试关注有符号乘法、极值和逐级位宽；FU 与父级测试加入背压、取消、复位、目的身份和 DIV 交互；整核协同验证进一步检查译码、架构状态及提交结果。

图2-12按测试输入、五层验证和期望与检查三列组织这些测试。算术测试输入同时驱动独立数学模型和算术 RTL，前者给出完整 128 位期望，与后者的实际值比较；分层事务用例分别驱动 VDot FU 和真实 VFEX4 父级，检查反压、取消、复位、身份，以及父级的互斥、切换和空闲关钟；AM 功能矩阵在全核 EMU 与 NEMU 上运行，通过 DiffTest 和程序自检核对架构状态。算术 RTL 的 10022 项输入与全核 117 个独立用例、124 次执行分别统计，各层证据独立归档。

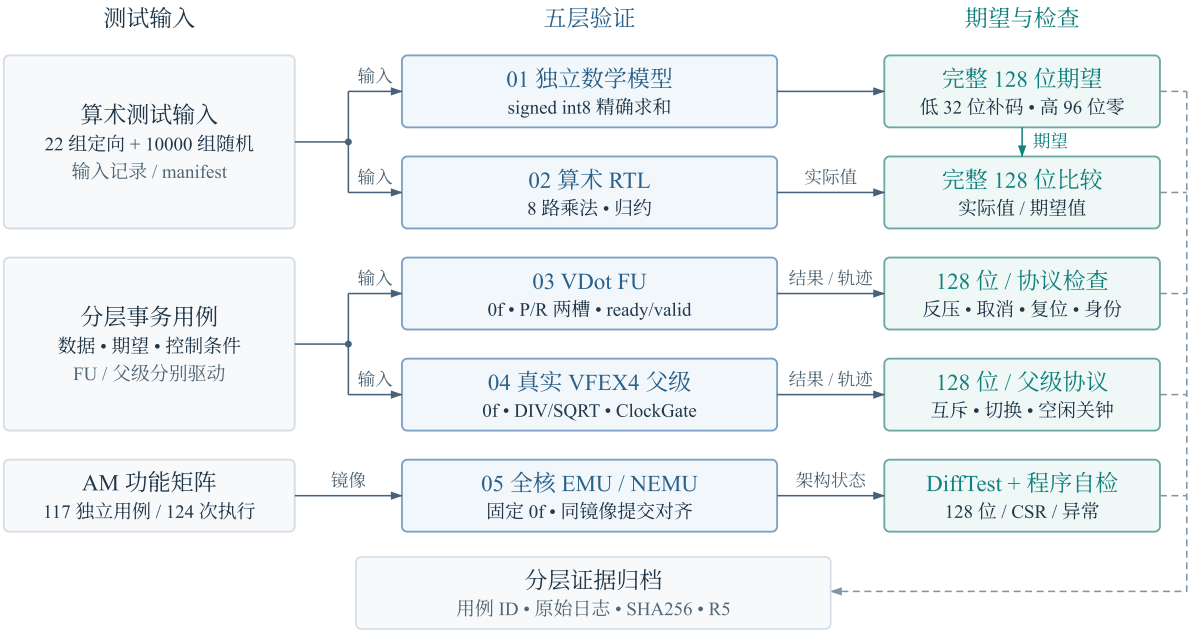


图 2-12　从数学结果到整核执行的验证层次

以下三类测试对应赛题 2.3.1 至 2.3.3。图2-13至图2-15的输入与响应取自固定八项两槽版本的全核观察记录，按 seq、ROB 和目的物理寄存器配对；第三阶段说明该版本的架构差分验证。

2.5.1 正数点积测试

两组有效输入均为 $[1, 2, 3, 4, 5, 6, 7, 8]$，八项平方和为 204。全核功能测试程序 functional-2 的运行记录中，实际完整 128 位结果为 0x000000000000000000000000000000cc，与预期一致。源高 64 位填入 A5 哨兵，用于检查硬件是否准确选择了低 8 个元素。图2-13摘录了这次响应的原始日志字段，除源值、预期与实际结果外，还记录了 seq=1、ROB=0:39、pdest=103 以及接受和响应周期。

```
[POSITIVE] functional-2  seq=1  ROB=0:39  pdest=103
vs1 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
vs2 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
expected sum      204
expected (128-bit)  0x000000000000000000000000000000cc
actual   (128-bit)  0x000000000000000000000000000000cc
ACCEPT cycle=339296 line=7030 | RESPONSE cycle=339298 line=7034
source: clean-02/fullcore-run-01/runs/functional-2/run.log
SHA256: 90b36b19e9d141e1b0bdad6bed461b4dbde4e534798f5fbdaa8819d9e2ef3211
```

图 2-13 正数点积：预期与实际结果均为 204

2.5.2 负数点积测试

有效输入为八个 -1 与八个 1，预期点积为 -8。`functional-0` 记录的完整结果为 0x00000000000000000000000fffffff8。低 32 位以二补码表达负数，高 96 位保持零，符合指令规定的结果布局。对应的日志摘录见图2-14，该响应记录为 seq=4、ROB=0:12、pdest=143。

```
[NEGATIVE] functional-0  seq=4  ROB=0:12  pdest=143
vs1 (128-bit)     0xa5a5a5a5a5a5a5a5ffffffffffffffff
vs2 (128-bit)     0xa5a5a5a5a5a5a5a501010101010101
expected sum      -8
expected (128-bit)  0x000000000000000000000000fffffff8
actual   (128-bit)  0x000000000000000000000000fffffff8
ACCEPT cycle=420291 line=7927 | RESPONSE cycle=420293 line=7931
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

图 2-14 负数点积：低 32 位保存负数补码

2.5.3 边界值点积测试

最大正结果由八个 -128 与八个 -128 产生。每项乘积为 16384，总和为 131072；`functional-0` 记录的完整结果为 0x00000000000000000000000000020000。这一输入检

验了乘法和归约在正端点的精确性。对应的日志摘录见图2-15，该响应记录为 seq=5、ROB=0:43、pdest=66。

```
[BOUNDARY] functional-0  seq=5  ROB=0:43  pdest=66
vs1 (128-bit)     0xa5a5a5a5a5a5a5a580808080808080808080
vs2 (128-bit)     0xa5a5a5a5a5a5a5a580808080808080808080
expected sum      131072
expected (128-bit)  0x00000000000000000000000000020000
actual   (128-bit)  0x00000000000000000000000000020000
ACCEPT cycle=448417 line=8400 | RESPONSE cycle=448419 line=8404
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

图 2-15　正端点：八对最小 INT8 数产生 131072

零输入测试采用八个 0 与八个 127，实际完整 128 位响应为零。另一数学边界由八个 −128 与八个 127 构成，预期总和为 −130048、低 32 位为 `0xfffe0400`；该值作为负端点的参考判据列入定向测试定义。

2.5.4　算术数据通路测试汇总

算术 RTL 测试包含 22 项定向输入和 10000 项随机输入，共 10022 项全部通过。定向输入针对符号、零值、极值和元素选择，随机输入进一步扩大数值组合覆盖。图2-16为测试程序输出的三行 PASS 汇总，描述的是算术数据通路；带背压和取消的时序行为在 FU 及父级测试中检查。

```
VDOT_DATAPATH_SUITE_PASS suite=directed count=22
VDOT_DATAPATH_SUITE_PASS suite=random count=10000
VDOT_DATAPATH_PASS directed=22 random=10000 total=10022
```

图 2-16　算术 RTL 的定向与随机测试汇总

2.5.5　响应配对与复查位置

四类已展示响应均按 seq、ROB 和 pdest 与接受记录配对，并比较完整 128 位结果。具体位置见表2-3，可与前述日志摘录中的输入和响应互相核对。零输入对应 seq=0、ROB=0:8、pdest=124。

表 2-3　已展示响应的日志位置

输入类型	日志	接受/响应周期	原始行号
正数	`functional-2/run.log`	339296 / 339298	7030–7034
负数	`functional-0/run.log`	420291 / 420293	7927–7931
最大正结果	`functional-0/run.log`	448417 / 448419	8400–8404
零输入	`functional-0/run.log`	311084 / 311086	5814–5818

原始记录保存在受测版本干净环境复现的运行归档 clean-02/fullcore-run-01/runs/中；本节展示执行单元响应，完整运行的架构差分结果见第三阶段。

图2-17把上述三类测试放在一起对照。三组输入的源高 64 位都填入 A5 哨兵，不参与运算；三个结果分别为 204、−8 和 131072，写回的高 96 位均为零，与期望一致。底部的 10022 项统计与图2-16是同一组算术测试。



图 2-17　正数、负数与边界测试的输入和结果

算术 RTL 之外，独立主机模型的 35 个测试方法也全部通过，输出见图2-18。该统计以测试方法为单位，与算术 RTL 的 10022 组输入分别计数。

在基础数值之外，测试还检查源高半区哨兵、旧目的哨兵、寄存器别名、v0 目的、非法配置和向量状态恢复。在 FU 和真实父级上，还向响应身份、写使能、取消状态和结果高位注入错误，确认检查器能够发现；正常运行时则核对完整响应和两槽排空。

2.6　设计取舍与扩展

2.6.1　两槽结构与结果保存

采用两槽结构后，相邻请求的乘法和归约可以重叠执行，连续的独立点积因此能每拍发起一笔。将 R 中的结果保存宽度收窄至 32 位，利用了高 96 位固定为零这一架构性质，减少了数据寄存器成本。每笔请求的 ROB、目的和控制信息则完整保留，以支持乱序执行、背压和精确取消。

共享父级的优化既要处理正常完成，也要处理取消后的资源释放。选择方案时同时比较功能、周期、标准单元面积和时序路径：曾评估过的一拍算术候选未通过父级的成本门槛，因此性能表仍以现有两槽实现为受测对象。面积和各类路径的具体数据见第三阶段。

仅运行主机模型检查，本命令不执行 RTL 或 NEMU



图 2-18 独立主机模型的测试输出

2.6.2 十六项 INT8 点积的功能进展

在八项实现的基础上，团队增加了独立十六项点积候选。该候选以补丁和配套验证快照交付，采用 `MATCH=0x0e00000b`、`MASK=0xfe00707f`，在 e8/m1、VL=16 时读取两个 128 位源的全部 16 个字节。八项指令继续使用原编码和 VL=8 语义。

候选复用两组八项乘积及归约模块。P 保存 16 个 16 位乘积，两组 19 位部分和相加得到 20 位精确和，再扩展为低 32 位结果。十六项和的范围为 $[-260096, 262144]$，R 仍保存 32 位，写回高 96 位为零。接收八项指令时，高八个乘积写零，使后级统一完成归约。两槽握手和取消机制沿用原结构。

2026 年 11 月 6 日，该候选完成了编译开关关闭和开启两种配置下的译码、FU 和真实父级验证。启用十六项扩展后，使用新 EMU 与匹配 NEMU 完成六例整核差分：2 例八项指令、4 例十六项指令全部通过。六例覆盖源高半区贡献、有符号正负端点、选定源目的别名、完整 128 位写回和恢复八项指令。相应 FU 与父级连续请求场景还核对了 2 拍首响应与 1 拍发起间隔。

该扩展的当前成果是指令与执行通路的功能验证。后续将通过完整模型调用、与较快 RVV 实现的性能对照，以及同条件下的综合与时序分析，评价其应用价值。本报告的周期和成本数据均来自八项受测版本。

2.6.3 元素位宽的参数化验证

另一组独立模块实验把输入元素扩展为 INT16，计算 8 项点积，并使用低 64 位结果字段、高 64 位清零的输出布局。该配置的最大正和为 2^{33}，精确累加至少需要 35 位。参数化乘法与归约模块完成 94 项定向输入和 10000 项随机输入的完整 128 位结果检查，并验证单请求包装中的反压、

取消与复位行为。这组实验说明，改变元素位宽时，可以按同样的方法重新确定中间位宽、结果字段和测试向量。

2.7 源码与证据入口

表2-4按内容列出本章涉及的源码与证据入口。各入口下保存完整源码版本、运行输入、原始

表 2-4 第二阶段源码与证据入口

内容	阅读入口
指令语义与编码	指令规格与定向向量
算术及两槽实现	八项弹性 FU 源码及乘积与归约源码
构建与版本选择	工程版本入口与可选配置
周期、面积与 STA	八项两槽受测版本的数据与原始报告
十六项扩展	候选结构、补丁与分层验证、六例整核差分结果
元素位宽参数化	八项 INT16 独立模块及完整结果验证

日志和失败记录，可用于复查实现细节与测量依据。

第三阶段：协同仿真与性能评估

3.1 从功能验证到工程评价

自定义指令需要在程序中产生正确结果，也需要与处理器的异常、状态和调度机制配合。本阶段使用独立 NEMU 参考模型和全核 DiffTest 核对架构行为，再以等价 RVV 程序比较完整测量窗口的周期。另外给出软件接入方式，以及综合面积和静态时序结果，用于评估这项设计的收益与成本。主要结果是：RTL 与 NEMU 参考模型在全部功能用例上协同仿真一致；相对等价的 RVV 程序，独立批量和依赖链场景分别加速 3.07 倍和 2.93 倍，单次点积加速 1.20 倍；8×32 GEMV 函数调用快 1.095 倍；VDot 功能单元的面积约占所在执行单元的 6.1%。

本阶段的数据都来自八项点积两槽实现的同一个固定版本（下文称“受测版本”），依次给出微基准、C8 软件调用和完整前向计算的结果，每组结果都注明输入和测量范围。

图3-1给出本阶段的协同仿真框架。同一 AM 镜像与共同启动 flash 分别加载到香山 DUT 和固定 NEMU 参考端；DUT 按架构提交推进参考模型，两端状态送入 DiffTest，比较完整 128 位向量结果、约定的向量 CSR 和异常状态。运行配置冻结为单核、VLEN128 和 128MB 内存，源码、二进制、配置、原始日志和真实退出码一并归档。两端的点积计算方式不同，参考端的具体处理见下节。

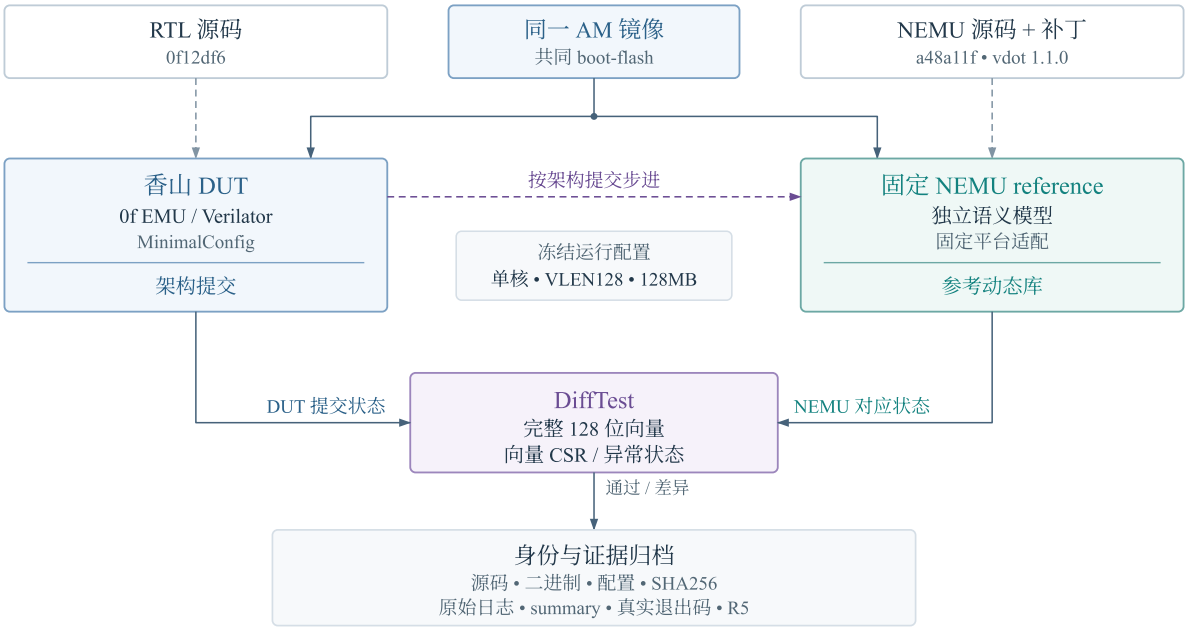


图 3-1 RTL 与独立参考模型的协同仿真

3.2 NEMU 参考语义与处理器接入

RTL 通过并行乘法、归约网络和流水状态执行点积；NEMU 参考模型则从指令字、源寄存器和向量配置出发，顺序计算八项有符号乘积之和。两者的内部实现方式不同，因此可以相互核对。

参考端首先匹配 custom-0 编码并检查合法配置，随后快照读取两个源寄存器，计算点积并构

造完整 128 位目的值：低 32 位为整数结果，高 96 位为零。快照读取使源、目的寄存器重合时仍使用本条指令执行前的数据。合法运算保留约定的向量状态；非法配置进入异常处理。

八项点积受测组合使用基于 NEMU 固定版本的 vdot 补丁及配套平台适配。适配使 NEMU 的 xTVAL 行为与该香山版本一致，异常码、陷阱 PC、xTVAL 和目的寄存器状态均保留检查。现有特权测试记录覆盖 M 模式。参考库、故障注入库与独立运行的 NEMU 各自记录构建来源和 SHA-256，复现时按用途选用。

图3-2展示参考端的处理流程。参考端先精确匹配 custom-0 编码，未匹配的指令交给其他译码；再检查向量状态是否合法，即 VS 已启用、vtype 合法、vl=8 且 vstart=0，非法时产生 cause=2 的非法指令异常且不写 vd；合法时先快照两源的低 64 位，执行 8 项有符号点积，再完整写回 128 位结果，并在成功后将 VS 置脏。

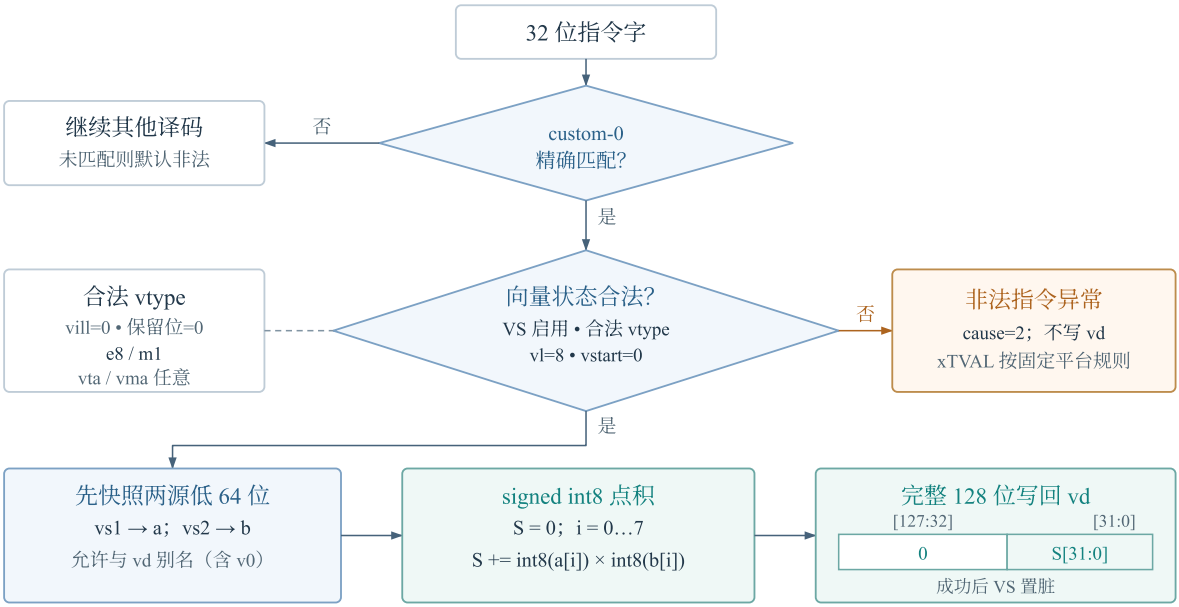


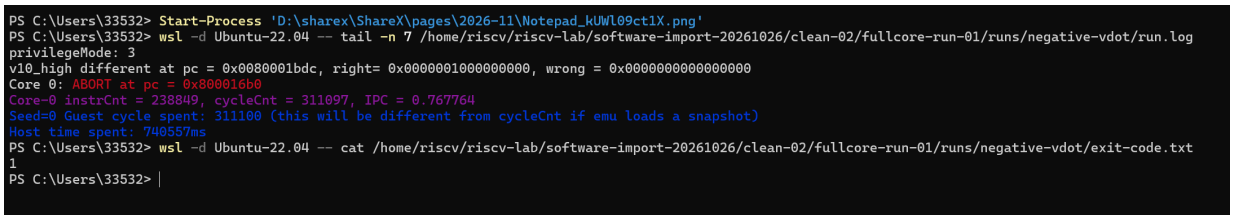
图 3-2　参考模型的译码、状态检查与完整写回

3.3　差分检验与错误检出

基础功能矩阵覆盖正数、负数、边界值、寄存器别名和非法向量配置。在受测版本上，全核矩阵包含 117 个不同用例，按矩阵分片共执行 124 次；数值正确性由算术 RTL 的 10022 项测试覆盖，握手控制由真实父级测试覆盖。最终验收时汇总了 13 项已有的全核记录和 1 项新增的控制诊断运行，并重新解析了它们的原始输出。

为了检验完整 128 位比较是否有效，负控在参考端的 vdot 结果中翻转 bit100。差分器检测到目的寄存器高半区不一致并按预期终止；恢复正常参考模型后，同一程序完成 15 项检查并到达 GOOD TRAP。图3-3给出了这两个互补结果：前者检验错误检出能力，后者检验正常执行路径。负控日志中可见 `v10_high different`、`ABORT` 和退出码 1，差异 PC 为 `0x80001bdc`，ABORT 报告的 PC 为 `0x800016b0`；日志中的 `right` 和 `wrong` 只是比较器两侧的标签，本例的扰动注入在参考端。正常日志中的退出码为 0。两段日志分别来自干净环境复现中的 `negative-vdot` 与 `functional-0` 运行。

（a）参考端翻转 bit100：差分器检出高位差异



（b）恢复正常参考模型：15 项检查通过

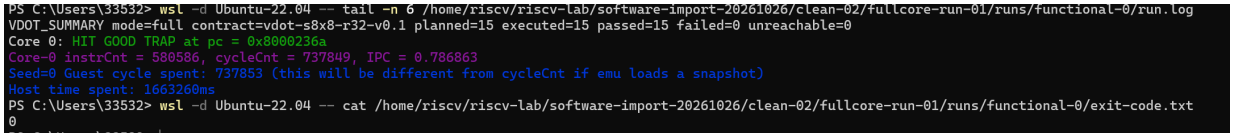


图 3-3 高位错误检出与正常参考模型恢复的原始日志

3.4 全核波形中的点积事务

为观察点积在全核中的输入、响应和提交，2026 年 11 月 5 日使用受测版本在干净环境复现中封存的生成 RTL，单独构建了支持 FST 的 EMU。1681 项生成输入与原清单一致；该模型运行原有的全核功能测试程序 functional-2，加载正常 NEMU 参考库并启用 DiffTest，最终到达 GOOD TRAP、退出码 0。以下波形将同一笔正数点积从执行输入关联到架构提交。

3.4.1 源数据、结果与身份

两个源寄存器均为 0xa5a5a5a5a5a5a5a50807060504030201，低八个有符号字节依次为 1 至 8。预期结果为

$$1^2 + 2^2 + \cdots + 8^2 = 204.$$

实际 128 位响应为 0x000000000000000000000000000000cc，符合低 32 位保存结果、高 96 位清零的布局。源的高 64 位填入 A5 作为哨兵，用来确认硬件只取了低 8 个元素。表3-1列出用于关联这笔事务的 ROB、物理目的、提交观察信号、PC 和机器字。

表 3-1 正数点积事务的关联字段

关联字段	观测值
ROB 与物理目的	ROB0:39；物理目的 103（十六进制 67）
提交观察	`io_valid=1`、`io_vecwen=1`；架构目的 v10
PC 与机器字	PC=`0x80001b8c`；机器字 `0x0294050b`

3.4.2 从有效区间到握手边沿

波形中的 valid 表示一段有效区间，事务接受则发生在 valid 与 ready 同时有效的时钟上升沿。表3-2把两者分开列出，便于与日志中的周期坐标对照。

同一事务的观察器日志记录接受、响应和内部 ROB 提交周期分别为 339296、339298、339301。接受到响应相隔两个全局模型周期，与第二阶段的两槽时序一致。内部 ROB 提交对应模型坐标

表 3-2 点积事务关键事件的波形坐标

事件	显示坐标（ps）	观察依据
输入有效（A）	678694	输入 valid、ready 均为 1，源值及身份有效
输入接受边沿	678696	边沿前握手有效，边沿后 P 槽有效
输出有效（B）	678698	R 槽有效，输出 valid、ready 均为 1，结果 204
输出握手边沿	678700	边沿前输出有效，边沿后 R 槽清空
DiffTest 提交观察（C）	678710	提交有效，PC、编码、ROB 与目的相符

678706，DiffTest 观察口在 678710 有效，反映两拍观察流水延迟。分析执行时延时使用实际握手和内部提交坐标，分析 DiffTest 接口时使用其观察坐标。

后面三幅截图中，上方为全局时钟，下方为点积单元的门控时钟，单元空闲时该时钟可以停止。显示单位 ps 是仿真模型的时间坐标，相邻全局周期相隔两个坐标单位。程序性能采用 DUT 周期，物理时序由综合和 STA 评估。

图3-4至图3-6三幅 GTKWave 截图分别展示输入、响应和提交。截图保留信号值，仅裁去层级栏和外部空白；完整原图、事务采样、运行回执与文件摘要见 evidence/vdot-wave-20261105/和 figures/source-screenshots/。

3.4.3 输入与接受

如图3-4所示，游标 678694 处输入 valid 和 ready 均为 1，两个源的低八字节为 1 至 8，ROB 值 27 和物理目的 67 以十六进制显示；678696 上升沿接收请求，此后 P 槽有效。

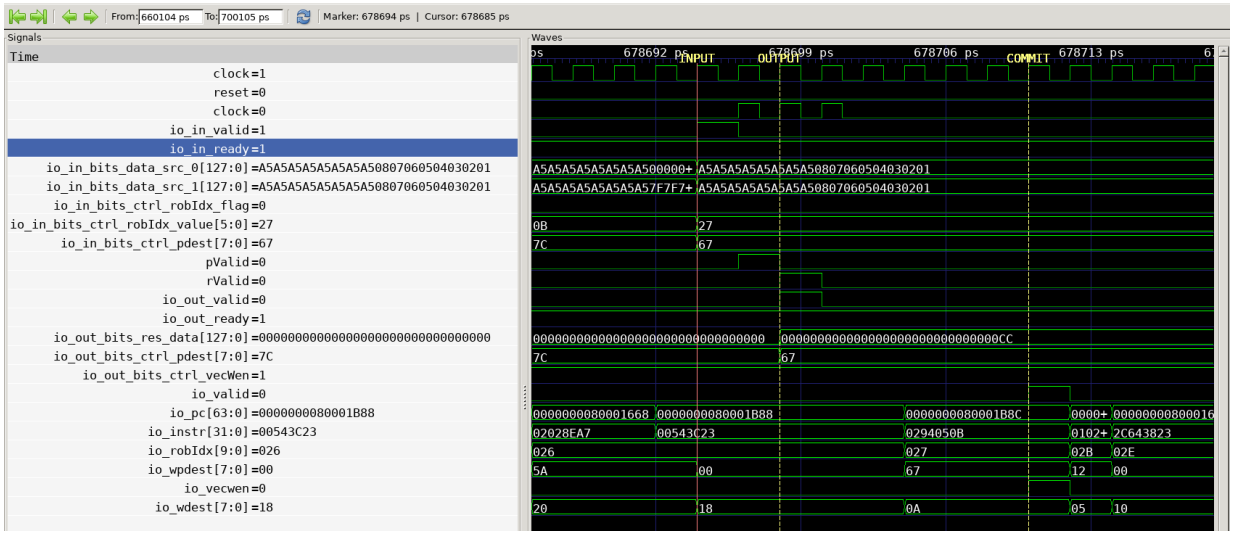


图 3-4 正数点积的输入与接受边沿

3.4.4 结果与响应

图3-5中，游标 678698 处 R 槽和输出 valid、ready 有效，结果为 204 且高 96 位为零；678700 边沿完成握手并释放 R 槽，此时提交观察口的 io_valid 仍为 0。

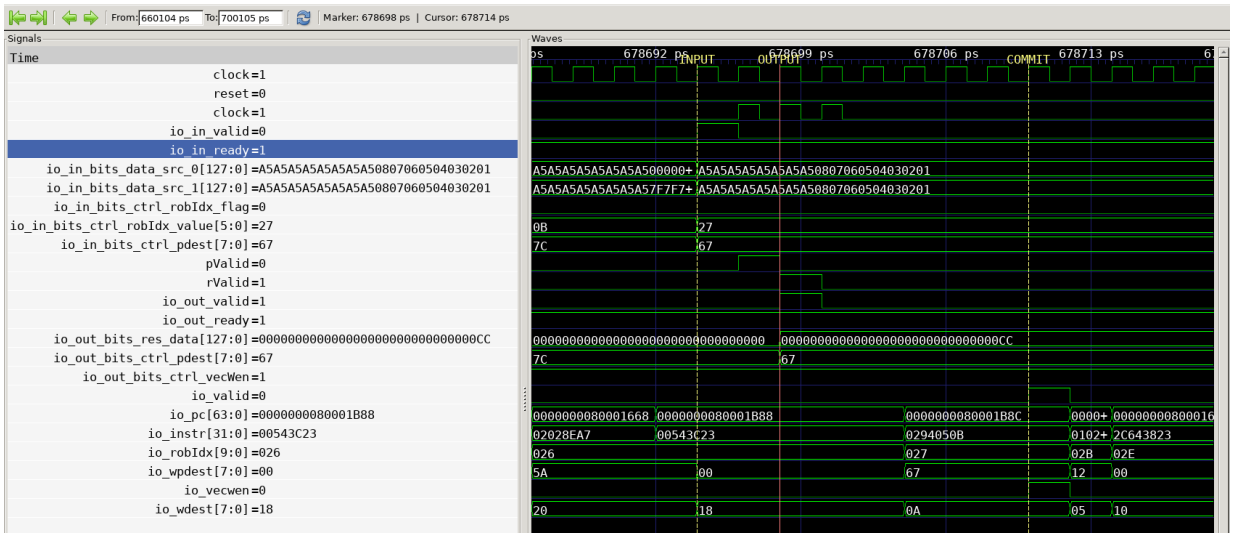


图 3-5 完整 128 位结果与响应握手

3.4.5 提交与架构目的

图3-6中，游标 678710 处提交有效，PC 为 80001B8C，机器字为 0294050B，ROB 为 027，物理目的为 67，架构目的为 0A（v10），向量写使能有效，均对应同一事务；观察口相对内部提交的两拍延迟已在前文说明。

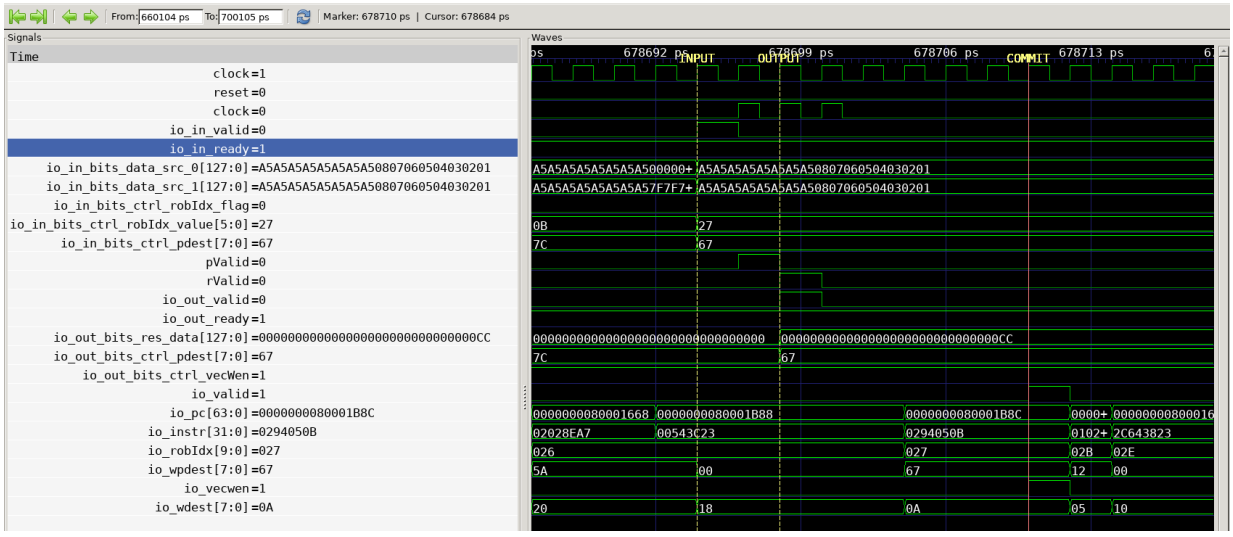


图 3-6 DiffTest 提交观察与架构目的

3.5 等价 RVV 对照与测量方法

8 项点积微基准中的标准 RVV 对照，通过宽乘和宽归约计算与 VDot 相同的有符号点积。单次已知配置序列先以 `vwmul.vv` 得到 16 位乘积，再切换到 `e16/m1/VL8`，清零整个目的寄存器并执行 `vwredsum.vs`，最后恢复入口配置。高 96 位在归约过程中保留为零，低 32 位保存完整整数和。

该微基准按使用场景组织：已知配置的单次点积为 5 条主体指令，通用入口增加 `vl/vtype` 保存，共 7 条；四个独立点积共享配置切换，主体合计 14 条。批处理时多个点积共用配置切换，标

准 RVV 也能充分利用独立计算；比较时双方实际执行的清零、数据搬运和状态管理都计入周期。

3.5.1 完整窗口与配对统计

测量采用同一输入、循环次数、镜像配置和边界规则。每个微基准场景使用三组数据，每组统计四个正式窗口；首次、预热和独立重放分别保存。计时保留实际执行的配置切换、循环与边界开销，统计单位为 DUT 全局周期。

香山是乱序处理器，计数器的读取时刻需要与指令执行位置对应。项目在固定受测实现上组合使用 fence 与 cycle、instret 读数，并通过观察器关联四条 CSR 指令的 PC、机器字、ROB 身份、采样和退休事件。cycle 差与全核采样 tick 差相符，instret 差与退休记录及 ELF 展开计数相符；计数器读数均按整数记录。这一方法只在本次受测硬件和测量规范下验证过。

图3-7按观察器事件绘出这一对应关系。四条 CSR 读取指令 I0、C0、C1、I1 分别标出进入 ROB、CSR 采样和退休三个时刻，横轴为相对 START 的全核 tick。cycle 采样差为 $136 - 65 = 71$，与 tick 差相等；instret 差为 12，按退休 count 求和，计数区间包含 I0 的退休、不包含 I1 的退休，12 项由 3 条 CSR、4 条 fence 和 5 条主体指令组成，主体中含 1 条 vdot。

加速比与周期降低率分别定义为

$$S = \frac{C_{\mathrm{RVV}}}{C_{\mathrm{VDot}}}, \qquad R = \frac{C_{\mathrm{old}} - C_{\mathrm{new}}}{C_{\mathrm{old}}}.$$

两者的分母不同，因此表中同时给出原始周期。完整窗口还包含调度、访存、循环和边界开销，不能与第二阶段 FU 的两拍最早响应直接比较。

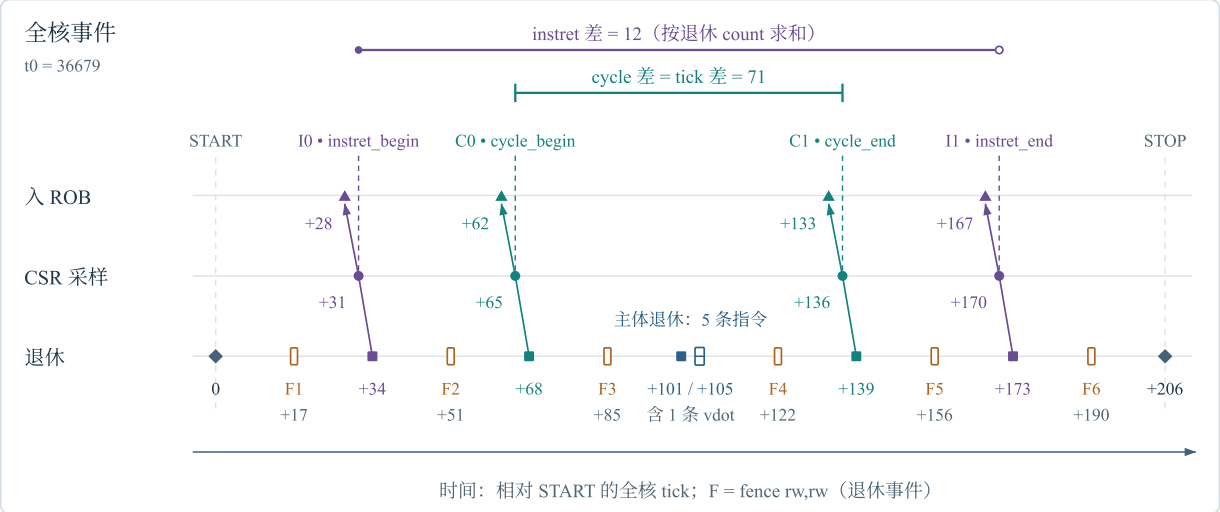


图 3-7 CSR 采样、退休与计时窗口的对应关系

3.6 硬件微基准结果

四类微基准分别观察单次点积、连续数据依赖、独立任务吞吐，以及加载、计算与存储共同作用下的表现。表3-3使用同镜像中的 VDot 与等价 RVV 路径；每个场景、每种实现使用三个数据集，每个数据集取四个正式窗口，共十二窗。

表 3-3 受测版本的微基准结果（周期均值）

场景	VDot	RVV	加速比
单次点积	71.00	85.25	1.2007×
依赖链（256 次）	1,857.00	5,440.50	2.9297×
四路独立批量（128 次）	221.00	678.00	3.0679×
加载、计算、存储流（64 次）	1,127.50	1,133.00	1.0049×

加速比按同一场景的 RVV 周期总和除以 VDot 周期总和计算。独立批量由 32 轮四路点积组成，能够持续向流水线提供相互独立的工作；依赖链则要求每次点积使用前次结果。对比这两个场景可以看出，任务之间是否存在数据依赖对收益影响很大。加入加载、存储与循环控制后，stream64 两条路径的周期接近，说明只加快点积本身，并不能明显缩短整个循环的时间。

与上一版单槽设计相比，两槽设计在同一镜像中把独立批量从 362.5 周期降到 221 周期，减少 39.03%。受测版本在加入 DIV 取消修复前后，同一输入的 96 对正式窗口周期完全相同。可见应用层面的收益还取决于任务独立性、数据搬运和循环开销。

图3-8以同场景 RVV 周期为 100% 进行归一化。VDot 在依赖链和独立批量场景中分别只需 RVV 周期的 34.13% 和 32.60%，单次点积为 83.28%，访存流为 99.51%，与 RVV 基本持平；这些比例与表3-3的加速比相对应。

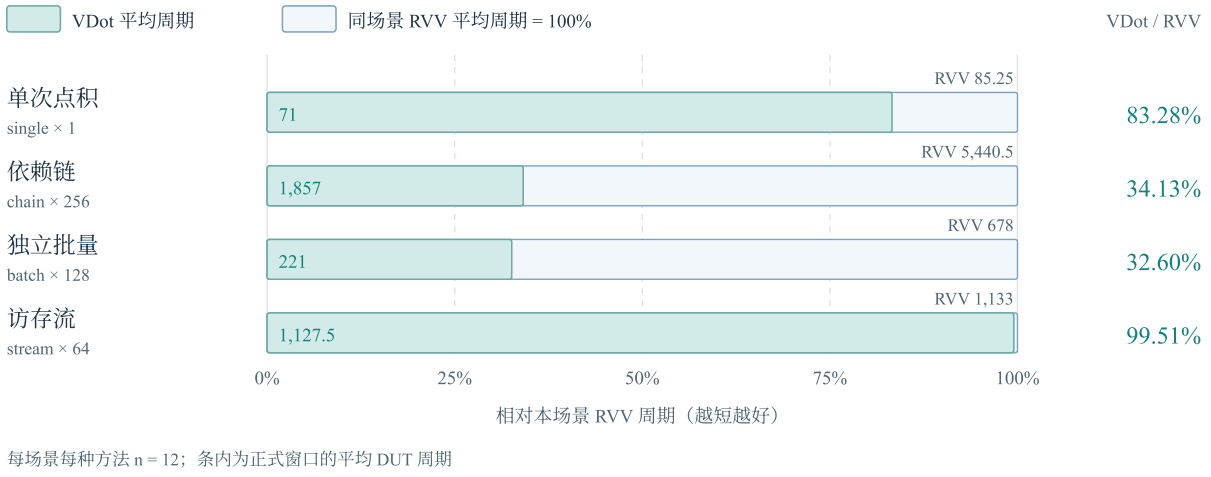


图 3-8 四类微基准周期与 RVV 归一化结果

3.7 软件调用与 C8 优化

前述微基准考察 8 项点积的指令序列，本节转向 8 行、每行 32 项 GEMV 的完整函数调用。硬件点积单元要在矩阵运算中发挥作用，还需要减少相邻计算之间的配置和数据搬运。首段初始化版本（firstdot）用首段点积直接初始化累加状态，随后接续其余分段；C8 版本在复用输入向量的 8 行、每行 32 项 GEMV 中，进一步消除重复的向量配置。两者分别减少累加初始化与配置管理的开销。

C8 提供普通 RV64 C 接口。调用者将行优先矩阵 $A[8][32]$ 和输入向量 $x[32]$ 连续存放，形成 288 字节输入；返回值写入 8 个 16 字节结果槽，每槽低 32 位保存 INT32 点积，高 96 位清零。槽

布局与指令的 128 位结果格式相同，结果可以直接用于功能检查和后续处理。

VDot 路径将每行分为四个 8 项点积段；RVV 路径则处理两个 16 项半组，以 `e8/m1/VL16` 宽乘和 `e16/m2/VL16` 宽归约接续得到完整 32 项组和。两种实现共享输入布局与完整结果格式，各自采用适合的分组和配置复用方式。三种实现的周期均值见表3-4。

表 3-4 8×32 整数 GEMV 调用的周期均值

8×32 整数 GEMV 调用	周期均值	对照意义
首段初始化版本	282.25	首段直接初始化
C8 配置复用 VDot	273.00	本节选用实现
同批较快合格 RVV	299.00	标准指令对照

在受测版本上的这组 8×32 配对测量中，输入已打包为 INT8，输出为 8 个完整结果槽。C8 相对首段初始化版本减少 3.2772% 的周期，相对较快 RVV 的加速比为 1.095238。统计采用三个数据集、每组四个正式窗口，包含完整调用开销。这些结果说明，整数点积之外的软件组织仍然影响最终收益。[1]

接入时，`xs_c8_gemv_8x32` 在编译期选择对应函数，显式 RVV 入口提供接口相同的对照版本。原合格程序、源码复制对照和 SDK 接入版的 31,440 字节 BIN 逐字节一致，装载段、48 项调用表和 12 组输入也一致。[2]由此可以确认，经过接口封装后实际执行的代码与测试时一致。新应用可以直接复用这些固定函数，但需要针对自己的数据准备流程和调用窗口重新验证。

图3-9概括了 C8 的软件入口和最终镜像核对。上半部分为普通 C 调用：288 字节的连续输入交给 `xs_c8_gemv_8x32` 或显式 RVV 入口处理，编译期开关决定前者连接 VDot 还是 RVV，输出为 8 个 16 字节结果槽，每槽低 32 位为结果、高 96 位清零；下半部分为原测试夹具的三方镜像等价核对，用于复用既有功能证据，本次没有新增全核运行。

3.8 软件接入与复现说明

3.8.1 固定平台与数据布局

C8 软件包面向本项目已验证的 RV64、lp64d、VLEN128 裸机环境，调用前启用向量状态。VDot 路径使用项目的自定义点积编码；RVV 路径使用标准向量指令。输入、输出各按 16 字节对齐并独立存放，输入在调用期间保持有效。接口约定列于表3-5。

函数沿用普通 RV64 C ABI 保存栈和被调用者保存寄存器。函数内部可以使用全部向量寄存器，返回配置固定为 `e8/m1/tu/mu`。结果槽宽度对应 VLEN128 下的整寄存器存储，因此调用方按照 16 字节步长读取各行结果。

3.8.2 编译期选择与调用

头文件 `selected_gemv_c8.h` 提供两个入口。`xs_c8_gemv_8x32` 默认连接配置复用 VDot；将 `XS_C8_GEMV_FORCE_RVV` 设为字面量 1 后，该名称连接 RVV 实现。

[1]C8 接口和结果见C8 软件包验收结果及其固定输入清单。这里的周期对应该 8×32 调用计划。

[2]三方最终镜像比较见C8 最终镜像等价结果。

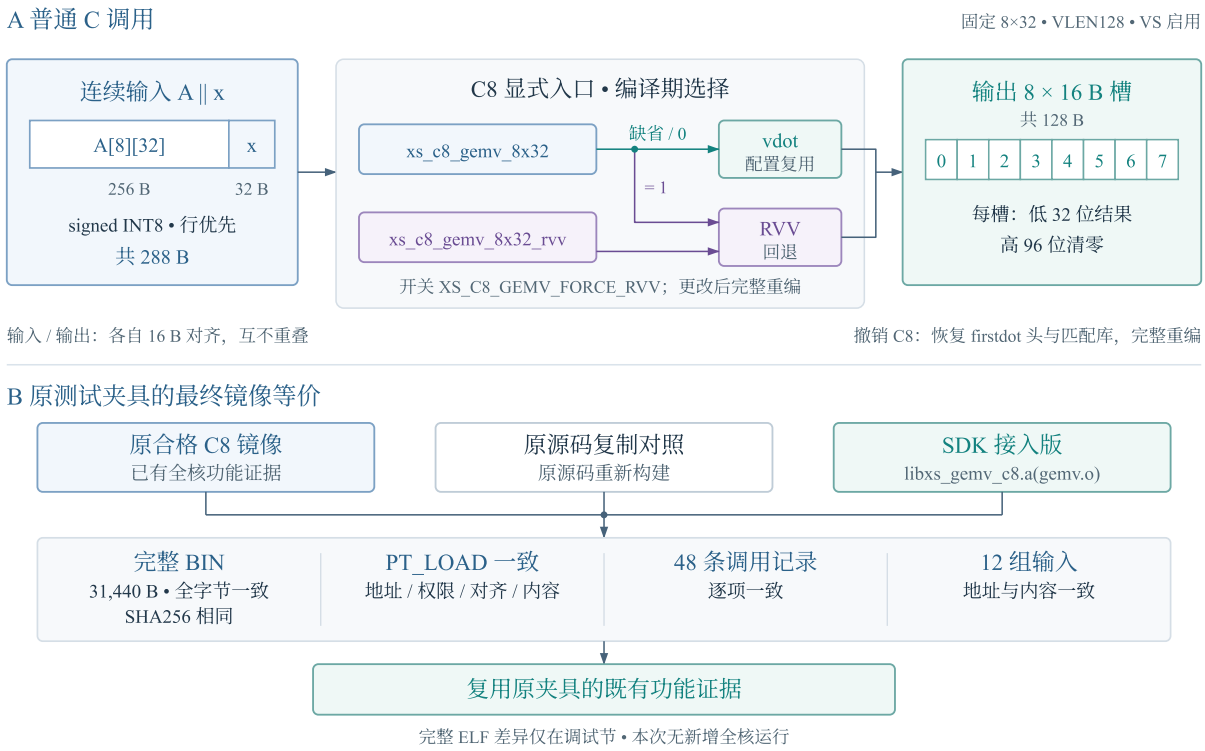


图 3-9 C8 软件入口、结果布局与最终镜像等价核对

表 3-5 C8 接口约定

项目	接口约定
输入	行优先 $A[8][32]$ 后接 $x[32]$，共 288 字节
输出	8 个 16 字节槽，共 128 字节
单槽	低 32 位 INT32 结果，高 96 位零
返回向量配置	`vl=8, vtype=0, vstart=0, vcsr=7`

`xs_c8_gemv_8x32_rvv` 始终连接 RVV。开关为 0 或 1，选择过程在预处理阶段完成。切换版本或开关后，重新编译相关翻译单元并链接匹配的静态库。

```
#include <stdint.h>
#include "selected_gemv_c8.h"

struct c8_slot {
    int32_t value;
    uint32_t upper_zero[3];
};
static _Alignas(16) int8_t packed_ax[288];
static _Alignas(16) struct c8_slot result[8];

/* Fill packed_ax with A[8][32] followed by x[32]. */
void run_c8(void) {
    xs_c8_gemv_8x32(packed_ax, result);
}
```

需要标准 RVV 对照时，可在同一调用位置使用显式 RVV 入口。保留首段初始化版本的应用

继续使用其对应头文件和库；C8 以独立名称加入软件包，便于按具体形状选择和回退。[3]

3.8.3 从函数对象到可执行镜像

软件包保留一个完整 `gemv.o`，由 `libxs_gemv_c8.a` 提供 16 个公共函数。固定 GNU 参数为`-march=rv64gcv -mabi=lp64d -mno-relax`。验证分别核对编译期别名、实际调用重定位、库成员抽取和最终装载内容，使每个公开入口都能追溯到被测试的函数。

SDK 接入的三方比较覆盖 BIN、`PT_LOAD`、BSS、函数地址与机器码、调用表及输入数据。三份 BIN 均为 31,440 字节且内容一致；去除调试节后的 ELF 也逐字节一致。调试信息因源目录和头文件路径不同而有差异，这部分单独保留，不影响运行内容的比对。图3-10给出完整的核对流程：627 项文件保护检查固定全部输入，随后从 `gemv.o` 抽取库成员并链接，比较三方最终 BIN、装载段、调用记录和输入；完整 ELF 的差异仅在调试节。右侧的 6 项结构与输入负控、1 项错误空库和 1 项调试正控均得到预期结果，由此得出的结论仅限于：原测试夹具已有的运行结果可以继续使用。[4]

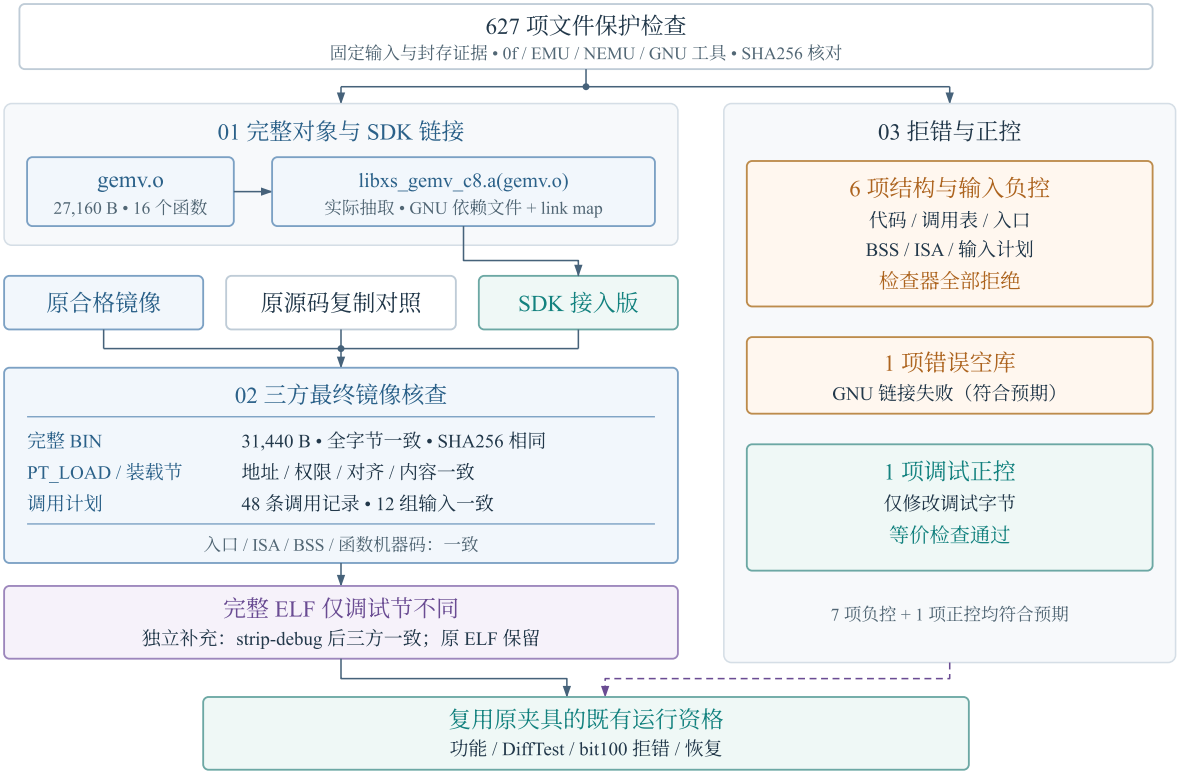


图 3-10 C8 接口、静态库与最终镜像的核对流程

复现时使用软件包的 README、固定输入清单及其配套核验工具，在独立输出目录保留构建结果。新应用接入时，应把模型准备、数据打包和计时窗口一并纳入验证，并在相同的数据准备和计时范围下比较 VDot 与较快的 RVV 实现。

[3]头文件及对齐示例见C8 编译期入口与同目录 `example.c`。接口细节见C8 接口规范。

[4]完整比较记录、链接映射及独立 GNU 复核见C8 最终镜像等价结果。

3.9 完整前向计算的组织与验证

本节验证 VDot 点积内核能否正确接入一个完整的模型计算：把量化矩阵向量乘内核接入一个小型 Transformer 的完整前向计算，分别使用标准 RVV 内核和 VDot 内核运行，并检查整个模型的计算结果。这里关注正确性，单次运行的周期不作为正式的性能结论。

3.9.1 从点积内核到模型计算链

应用验证将量化矩阵向量乘接入一个六层 Transformer 的完整 forward。固定合成模型的模型维度为 288，前馈中间维度为 768，注意力头数为 6，词表大小为 32,000；测试在规定初始化后，执行 token=1、position=0 的一次前向计算。权重与尺度预先完成打包，该准备成本不计入本次窗口。计数窗口覆盖一次完整 forward 调用，包含激活量化；计算依次经过嵌入读取、各层归一化、Q/K/V 投影、位置编码、注意力、输出投影及前馈网络，最终生成 32,000 项分类器输出。

本节的量化 GEMV 内核按每 32 项一组计算，直接生成 FP32 行输出。每组先求精确的 INT32 点积，再转换为 FP32，依次乘以权重尺度和激活尺度，最后按组号顺序累加到输出。不同内核共用权重、尺度数组及这一浮点计算顺序。Q、K、V 投影复用同一份量化激活，前馈网络的两个输入投影也共享量化结果。

3.9.2 工作量与数据布局

表3-6列出一次 forward 中的 43 个量化 GEMV。这里以一项乘加记一个 MAC，统计矩阵的有效维度；注意力、归一化、激活函数等运算不计入该表，但包含在完整 forward 中。

表 3-6 固定模型一次 forward 的量化 GEMV 工作量

矩阵类别	数量	每个矩阵的形状	逻辑 MAC	占比
Q/K/V/O 投影	24	288×288	1,990,656	13.11%
前馈 W1/W3	12	768×288	2,654,208	17.48%
前馈 W2	6	288×768	1,327,104	8.74%
分类器	1	$32,000 \times 288$	9,216,000	60.68%
合计	43	—	15,187,968	100.00%

分类器占逻辑 MAC 的 60.68%，因此大矩阵路径最值得优化。这个比例描述乘加工作量；其实际耗时还取决于访存、尺度处理、循环控制及处理器执行资源。

现有布局将权重按八行和量化组打包，尺度沿每个输出行连续存放。输入维度为 288 的矩阵由 288 行面板处理，共调用 172 个面板；输入维度为 768 的六个 W2 矩阵使用对应内核。最后一个面板不足时仍按完整面板计算，只复制有效输出，因此多计算了 1,408 个填充行，使实际整数乘加项为 15,593,472，比有效工作量多 2.67%。

3.9.3 完整计算链的验证结果

固定模型已分别使用标准 RVV 内核（下称 FR）和 VDot 内核（下称 TA）在香山仿真模型上完成完整前向计算验证。两路均完成 3,691,424 字节的全状态比较，并核对输出保护区、调用状态

及浮点异常标志。表3-7是这次正确性实验中两条路径各一次的计数窗口；TA 独立重放得到相同的完整验证结果和窗口计数。

表 3-7 完整 forward 正确性实验中的单窗口目标机观测

实现	周期数	退休指令数
FR：标准 RVV	20,258,039	7,502,872
TA：自定义 VDot	19,774,245	10,122,100

从表中可以看到，TA 退休的指令更多，周期却更少，因此退休指令数不能代替周期来比较性能。这两次都是单窗口记录，只用于检查完整执行和复现，稳定的性能比较还需要更多测量。

3.10 综合面积、时序与设计成本

设计成本使用 Nangate45 Typical 库，条件为 1.10V、25 °C，工具为 Yosys 0.9、ABC 1.01 和 OpenSTA 2.0.17。受测对象是带 32 位结果保存和 DIV 取消修复的八项点积两槽版本。主档采用 1 ns 时钟周期、5 fF 负载及固定线负载模型，另以 0.5 ns、2 ns 周期和 1 fF、20 fF 负载检查敏感性。主档下的面积与时序结果见表3-8。

表 3-8 VDot FU 与 VFEX4 父级的面积与时序

指标	独立 VDot FU	共享 VFEX4 父级
标准单元面积（μm^2）	17298.246	284614.946
寄存器至寄存器到达时间（ns）	1.163404	3.261218
该类路径 setup 裕量（ns）	−0.240117	−2.341112
所报告路径最差 setup 裕量（输入到寄存器，ns）	−0.268520	−2.393998

两个统计范围具有包含关系：VFEX4 覆盖 VDot、DIV/SQRT 及共享控制。最差 setup 裕量来自输入至寄存器路径，与表中的寄存器至寄存器路径分别列示。1 ns 主档下两种统计范围均存在 setup 违例，后续物理实现需要继续进行时序收敛；这些综合和 STA 数据用于在相同工艺和约束下比较不同结构的成本。

理解这些负裕量需要两点背景。第一，赛题没有规定时序门槛；Nangate45 是开源的 45 nm 教学工艺库，1 ns（1 GHz）是本项目自定的较紧约束。放宽到 2 ns 时，VDot 功能单元的全部路径都满足时序，父级中 VDot 内部的输入到寄存器、寄存器到寄存器路径也有正裕量（分别为 +0.45 ns 和 +0.57 ns）。第二，父级最差的寄存器到寄存器路径和时钟门控使能路径都位于原有的浮点除法单元内部，并非 VDot 的运算逻辑。与 VDot 相关、仍未收敛的主要是其结果经共享输出逻辑到达父级输出端口的路径，1 ns 和 2 ns 下的裕量分别为 −1.93 ns 和 −0.93 ns，这是后续时序优化的重点。

将结果状态从 128 位收窄为 32 位后，独立 FU 面积相对其前一版减少 2.33386%，共享父级减少 0.137349%。进一步加入 DIV 取消修复时，独立 FU 面积保持 17298.246 μm^2，父级由 284879.882 降至 284614.946 μm^2；对应 96 对正式微基准窗口的周期逐对相同。

图3-11给出面积构成和全部 14 类路径的 setup 裕量，其中 i2o、i2r、r2o、r2r 分别表示输入到输

出、输入到寄存器、寄存器到输出和寄存器到寄存器的路径，带 vdot 前缀的是父级中与 VDot 相关的路径。VDot FU 约占父级面积的 6.1%；在 1 ns 主档下，FU 的 i2r、r2r 两类路径和父级除 ready 以外的 8 类路径为负，共 10 类为负值，其中父级 i2o、i2r、r2o、r2r 四类路径的裕量约为 −2.3 ns。

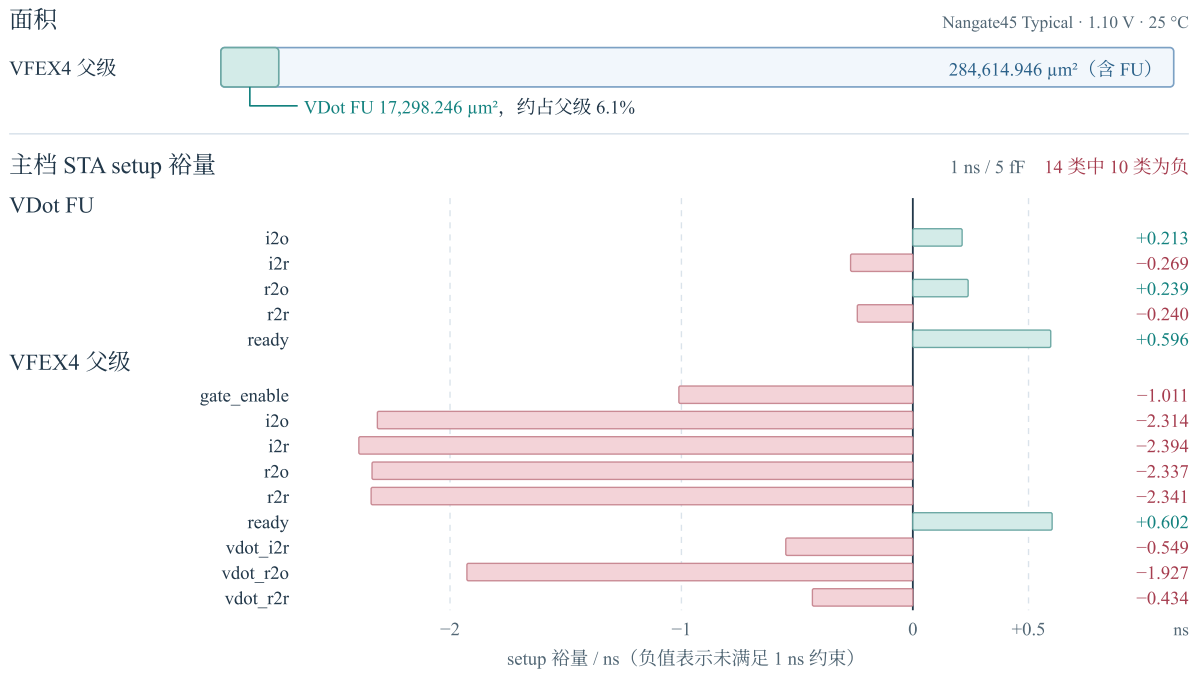


图 3-11　Nangate45 模块面积与主档 STA 裕量

3.11　设计取舍与应用方向

两槽流水提高了独立点积的发起速率；结果只保存 32 位，利用的是结果高 96 位恒为零；DIV 取消修复使共享资源能在取消后及时释放。软件侧则通过首段初始化、C8 配置复用及更适合量化组的 RVV 组织减少实际调用开销。这些优化作用在不同环节，通过配对实验可以把各自的收益和代价分开衡量。

固定 GEMV 已经给出普通 C 调用的周期结果，完整前向计算则验证了量化内核能正确接入整个模型的计算。最新十六项点积候选完成了指令和执行通路的功能验证，后续应用比较将继续保留较快的标准 RVV 对照。布局后时序、整核频率和功耗需要在后续物理实现中评估。

图3-12汇总固定八项受测组合在各层级已有和待补的证据。功能正确性和周期观察在四个层级都有记录；单元面积和布局前 STA 只覆盖局部 FU 和 VFEX4 父级，且 STA 尚未收敛；功耗以及整核层级的物理指标仍待评估。FU 的握手拍与整核、C8 的 DUT 周期分别计量。

3.12　补充：三种标准 RVV 实现的完整前向观测

本节记录一组只涉及标准 RVV 实现的补充实验，说明软件组织方式对完整前向计算的影响。它不包含 VDot 指令，因此不用于评价 VDot 的收益。

在相同模型与共同程序上，进一步接入了两种标准 RVV 软件组织：GS32 一次处理完整的 32 项量化组；paired16 把前后半组对应的乘积在 16 个 INT16 通道中相加，再扩宽归约。paired16 在



图 3-12　八项点积受测组合的验证与评估范围

入口检查激活值，遇到 −128 时执行完整 FR 路径，保持完整的 signed-int8 数值域。入口检查和回退的开销也计入调用。

三路共同程序已完成原 NEMU 参考验证，并于 2026 年 11 月 6 日完成三路香山目标运行。三种方法共用 ELF（SHA256 前缀 `be6c3ead424d`），目标镜像仅四字节方法选择字不同；W2 继续使用 FR，实现差异集中在输入维度为 288 的矩阵路径。三个目标运行均通过原 DiffTest、3,691,424 字节完整状态比较、保护区与调用状态检查，以及精确浮点异常标志检查（flags=1）。四个 CSR 读取的采样与退休链均通过核验；三条标准 RVV 路径都没有执行自定义点积指令，符合预期。

表3-9列出同一构建的三个完整 forward 目标窗口；目标退休指令数与各自已有 NEMU 参考记录相同。

表 3-9　共同 ELF 的完整 forward 单窗口目标观测

实现	目标窗口周期数	目标窗口退休指令数	相对 FR 周期减少
FR	22,244,288	7,502,886	—
GS32（G）	19,996,570	4,828,788	10.1047%
paired16（P）	17,382,632	5,175,371	21.8558%

在本次固定窗口中，paired16 比 GS32 少用 13.0719% 的周期，同时多退休 346,583 条指令，即增加 7.1774%。因此不能只看指令数来判断两种实现谁更快。

本组使用固定合成模型，计时范围为共同 prepare 之后、position=0、token=1 的一次完整 forward，不包含模型生成或载入、离线 packing、程序启动及此前的初始化和 prepare。参考验证另外包含 paired16 独立重放与末项输出扰动负控；本轮目标运行只有三个正常观测，没有新增目标重放或负控。

本轮只确认了上述范围内目标机运行结果正确，尚未完成正式的性能测量，也没有逐条审计整个退休窗口。表中百分比只是这一次单窗口的计算结果；三路都是标准 RVV 实现，也不能用来说明 VDot 指令的收益。前述 FR/TA 实验和 C8 实验各自使用不同的程序与测量窗口，不能与本表交叉计算加速比。

3.13 评估对象与证据索引

报告按实际受测对象组织结果。四类微基准、普通 C 调用和完整 forward 各自对应固定的代码、程序与输入；可沿表3-10定位产生每组结果的实验。

表 3-10 评估对象与版本标识

评估对象	版本标识	对应结果
硬件微基准	受测版本：硬件提交 `0f12df6753` 及对应 EMU	四类微基准的正式窗口和同镜像比较
干净环境复现	`clean-02` 构建及其参考模型	干净环境构建、差分和工程验收
C8 普通 C 入口	最终 BIN 的 SHA256 前缀 `56805506eb98`	固定 GEMV、调用状态与最终镜像等价
完整前向（FR 与 TA）	同一套仿真环境（硬件提交 `34bae6d4`）与两种调用程序	首个 token 完整执行、全状态比较及目标机单窗口观测
三种 RVV 实现的完整前向	同一套仿真环境、共同 ELF `be6c3ead424d` 及固定模型	完整状态比较、参考重放与负控，以及三个正常目标窗口的周期和退休指令观测

四类微基准可由微基准原始窗口 CSV与测量方法与成本汇总复查；软件包的使用与版本选择见C 入口与版本间比较。

完整前向计算的FR/TA 目标机记录和FR/GS32/paired16 共同参考结果分别对应各自的实验；本次三路 RVV 目标观察见三路目标结果及证据入口。前述两个摘要和本次三路原始结果随 LaTeX 整合包提供；后者保存为 `first-forward-target-results.json`，来源提交、归档成员及 SHA256 见同目录的 `provenance.json`。各实验目录同时保存构建信息、输入清单和原始运行记录。

3.13.1 复现与图像来源

本报告的实验资料截至 2026 年 11 月 6 日，文字和插图于 11 月 8 日修订。产生测量数据的构建已在表3-10中单独标识，仓库合并和文档编辑不影响这些实验数据。

原始日志、失败记录及构建清单随各实验目录保存。最终验收保留原 UTC 占用审计记录，并使用同一启动环境下的单调时钟记录补充核对；性能表使用 DUT 周期。复现者可从各目录的 RESULT、测量规范和输入清单找到对应的原始记录。

技术图依据源码、测试记录及测量数据绘制，终端与 GTKWave 截图保留实际观察内容。LaTeX 整合包包含正文直接引用的图像与日志，完整工程证据按本节索引在团队仓库及对应归档中查询。